

座・Navi（za-navi）転記用資料

Java Servlet/JSP（MVC2） 座席予約管理システム ソースコード一式 2026-06-07 時点

1. entity（エンティティ：DBの1行を表すクラス）

java/entity/entity/Account.java

```
package entity;

/**
 * 社員アカウント1件分のデータを表すentity（DBの accounts テーブル1行に相当）。
 * MVC2では「Model」のうち、データそのものを表す部分。
 * DAOがDBやモックデータから取り出した値をこのクラスに詰め替えて、Logic・JSPへ渡す。
 */
*
*   userId   : ログインID（社員番号など）
*   password : ログインパスワード
*   name      : 氏名
*   bId       : 所属部署ID（部署名そのものではなくID。名前は LocationsDAO.getBNameById(bId) で取得する）
*   admin     : 権限種別（0=一般ユーザー，1=管理者）
*/
public class Account {
    private String userId;
    private String password;
    private String name;
    private int bId;
    private int admin; // 0=一般，1=管理者

    public Account() {}

    public Account(String userId, String password, String name, int bId, int admin) {
        this.userId = userId;
        this.password = password;
        this.name = name;
        this.bId = bId;
        this.admin = admin;
    }

    public String getUserId() { return userId; }
    public void setUserId(String userId) { this.userId = userId; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getBId() { return bId; }
    public void setBId(int bId) { this.bId = bId; }
    public int getAdmin() { return admin; }
    public void setAdmin(int admin) { this.admin = admin; }
}
```

java/entity/entity/Location.java

```
package entity;

/**
 * 「部署マスタ」1件分のデータを表すentity（DBの locations/departments テーブル1行に相当）。
 *
 * ※クラス名は Location（場所）だが、中身は「部署ID（bId）と部署名（bName）」であり、
 * 座席の「拠点（三田オフィス・芝浦オフィスなど）」とは別物なので注意。
 * "b" は「部署 = bumon」の頭文字。
 *
 * bId    : 部署ID（Account.bId と対応する）
 * bName  : 部署名（画面に表示する文字列。例:「営業部」「開発部」）
 */
public class Location {
    private int bId;
    private String bName;

    public Location() {}

    public Location(int bId, String bName) {
        this.bId = bId;
        this.bName = bName;
    }

    public int getBId() { return bId; }
    public void setBId(int bId) { this.bId = bId; }
    public String getBName() { return bName; }
    public void setBName(String bName) { this.bName = bName; }
}
```

```

package entity;

/**
 * 予約・行先登録1件分のデータを表すentity (DBの reservations テーブル1行に相当)。
 * 「出社」「在宅」「出張」のいずれであっても、すべてこの1つのクラスで表す
 * (行き先の種類によって座席ID(seatId)があったり0だったりする、という違いだけ)。
 *
 *   reserveId   : 予約ID (一意の連番)
 *   userId      : 予約の本人 (誰の予定か)
 *   registerId  : 予約を登録した人 (自分で登録ならuserIdと同じ。管理者の代理予約なら管理者のID)
 *   reserveDate : 対象日 (YYYY-MM-DD形式の文字列)
 *   gyosaki     : 行先の種類 ("出社" / "在宅" / "出張" のいずれか)
 *   memo        : 自由記入のメモ
 *   seatId      : 予約した座席ID (0の場合は座席なし=在宅・出張など)
 *   cancelFlag  : 取消状態 (0=有効な予約, 1=取消済み)
 *   cancelDate  : 取消した日付 (YYYY-MM-DD)。取消していなければ null
 */
public class Reservation {
    private int reserveId;
    private String userId;
    private String registerId;
    private String reserveDate;
    private String gyosaki;
    private String memo;
    private int seatId;
    private int cancelFlag;
    private String cancelDate;

    public Reservation() {}

    public Reservation(int reserveId, String userId, String registerId,
                       String reserveDate, String gyosaki, String memo,
                       int seatId, int cancelFlag, String cancelDate) {
        this.reserveId = reserveId;
        this.userId = userId;
        this.registerId = registerId;
        this.reserveDate = reserveDate;
        this.gyosaki = gyosaki;
        this.memo = memo;
        this.seatId = seatId;
        this.cancelFlag = cancelFlag;
        this.cancelDate = cancelDate;
    }

    public int getReserveId() { return reserveId; }
    public void setReserveId(int reserveId) { this.reserveId = reserveId; }
    public String getUserId() { return userId; }
    public void setUserId(String userId) { this.userId = userId; }
    public String getRegisterId() { return registerId; }
    public void setRegisterId(String registerId) { this.registerId = registerId; }
    public String getReserveDate() { return reserveDate; }
    public void setReserveDate(String reserveDate) { this.reserveDate = reserveDate; }
    public String getGyosaki() { return gyosaki; }
    public void setGyosaki(String gyosaki) { this.gyosaki = gyosaki; }
    public String getMemo() { return memo; }
    public void setMemo(String memo) { this.memo = memo; }
    public int getSeatId() { return seatId; }
    public void setSeatId(int seatId) { this.seatId = seatId; }
    public int getCancelFlag() { return cancelFlag; }
    public void setCancelFlag(int cancelFlag) { this.cancelFlag = cancelFlag; }
    public String getCancelDate() { return cancelDate; }
}

```

```
    public void setCancelDate(String cancelDate) { this.cancelDate = cancelDate; }  
}
```

java/entity/entity/Seat.java

```
package entity;  
  
/**  
 * 座席1件分のデータを表すentity (DBの seats テーブル1行に相当)。  
 * 「拠点 → フロア → エリア → 席名」の4段階で、座席の場所を一意に表す。  
 * 例: baseName="三田オフィス", fName="2F", areaName="Aエリア", seatName="A-01"  
 */  
public class Seat {  
    private int seatId;  
    private String baseName; // 拠点  
    private String fName;    // フロア  
    private String areaName; // エリア  
    private String seatName; // 席番号  
  
    public Seat() {}  
  
    public Seat(int seatId, String baseName, String fName, String areaName, String seatName) {  
        this.seatId = seatId;  
        this.baseName = baseName;  
        this.fName = fName;  
        this.areaName = areaName;  
        this.seatName = seatName;  
    }  
  
    public int getSeatId() { return seatId; }  
    public void setSeatId(int seatId) { this.seatId = seatId; }  
    public String getBaseName() { return baseName; }  
    public void setBaseName(String baseName) { this.baseName = baseName; }  
    public String getFName() { return fName; }  
    public void setFName(String fName) { this.fName = fName; }  
    public String getAreaName() { return areaName; }  
    public void setAreaName(String areaName) { this.areaName = areaName; }  
    public String getSeatName() { return seatName; }  
    public void setSeatName(String seatName) { this.seatName = seatName; }  
}
```

2. dao（データアクセス：モック版。後でMySQL接続に差し替え予定）

java/dao/dao/AccountsDAO.java

```
package dao;

import entity.Account;
import java.util.*;
import java.util.stream.Collectors;

/**
 * ユーザーアカウントDAO（モック版）
 * TODO: 後でMySQL接続に差し替え
 * - findById(userId)      → SELECT * FROM accounts WHERE user_id = ?
 * - findAll()             → SELECT * FROM accounts
 * - authenticate()        → SELECT * FROM accounts WHERE user_id=? AND password=?
 */
public class AccountsDAO {

    // TODO: 後でMySQL接続に差し替え - この静的リストをDBクエリ結果に置き換える
    // ログイン動作確認用：一般ユーザーは user001~user005／パスワードは pass001~pass005、
    // 管理者は admin001／パスワードも admin001
    private static final List<Account> ACCOUNTS = Arrays.asList(
        new Account("user001", "pass001", "田中太郎", 1, 0),
        new Account("user002", "pass002", "鈴木花子", 1, 0),
        new Account("user003", "pass003", "佐藤次郎", 2, 0),
        new Account("user004", "pass004", "高橋美咲", 2, 0),
        new Account("user005", "pass005", "伊藤健一", 3, 0),
        new Account("admin001", "admin001", "管理者山田", 3, 1)
    );

    public Account findById(String userId) {
        // TODO: 後でMySQL接続に差し替え: SELECT * FROM accounts WHERE user_id = ?
        return ACCOUNTS.stream()
            .filter(a -> a.getUserId().equals(userId))
            .findFirst().orElse(null);
    }

    public Account authenticate(String userId, String password) {
        // TODO: 後でMySQL接続に差し替え: SELECT * FROM accounts WHERE user_id=? AND password=? (パスワードはハッシュ化推奨)
        return ACCOUNTS.stream()
            .filter(a -> a.getUserId().equals(userId) && a.getPassword().equals(password))
            .findFirst().orElse(null);
    }

    public List<Account> findAll() {
        // TODO: 後でMySQL接続に差し替え: SELECT * FROM accounts ORDER BY user_id
        return new ArrayList<>(ACCOUNTS);
    }

    public List<Account> findByName(String keyword) {
        // TODO: 後でMySQL接続に差し替え: SELECT * FROM accounts WHERE name LIKE ?
        if (keyword == null || keyword.isEmpty()) return new ArrayList<>(ACCOUNTS);
        return ACCOUNTS.stream()
            .filter(a -> a.getName().contains(keyword))
            .collect(Collectors.toList());
    }

    public List<Account> findByBId(int bId) {
        // TODO: 後でMySQL接続に差し替え: SELECT * FROM accounts WHERE b_id = ?
        return ACCOUNTS.stream()

```

```
        .filter(a -> a.getBId() == bId)
        .collect(Collectors.toList());
    }
}
```

java/dao/dao/LocationsDAO.java

```
package dao;

import entity.Location;
import java.util.Arrays;
import java.util.List;

/**
 * 部署マスタDAO（モック版）。
 * ※クラス名・entity名は Location（場所）だが、扱っているのは「部署（営業部・開発部など）」であり、
 * 座席の「拠点（三田オフィスなど）」とは別物なので注意（entity.Location のコメントも参照）。
 *
 * TODO: 後でMySQL接続に差し替え
 * - findAll()    → SELECT * FROM departments ORDER BY b_id
 * - findById()   → SELECT * FROM departments WHERE b_id = ?
 */
public class LocationsDAO {

    // TODO: 後でMySQL接続に差し替え - この静的リストをDBクエリ結果に置き換える
    private static final List<Location> DEPARTMENTS = Arrays.asList(
        new Location(1, "営業部"),
        new Location(2, "開発部"),
        new Location(3, "管理部")
    );

    public List<Location> findAll() {
        // TODO: 後でMySQL接続に差し替え: SELECT * FROM departments ORDER BY b_id
        return DEPARTMENTS;
    }

    public Location findById(int bId) {
        // TODO: 後でMySQL接続に差し替え: SELECT * FROM departments WHERE b_id = ?
        return DEPARTMENTS.stream().filter(d -> d.getBId() == bId).findFirst().orElse(null);
    }

    public String getBNameById(int bId) {
        Location loc = findById(bId);
        return loc != null ? loc.getBName() : "不明";
    }
}
```

java/dao/dao/MastersDAO.java

```
package dao;

import entity.Seat;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/**
 * 座席マスタCRUD用DAO（モック版）
 * SeatsDAO.SEATS（静的リスト）を共有して操作する。
 *
 * TODO: 後でMySQL接続に差し替え
 * - getAll()      → SELECT * FROM seats ORDER BY seat_id
 * - add(seat)     → INSERT INTO seats (base_name, f_name, area_name, seat_name) VALUES (...)
 * - update(seat)  → UPDATE seats SET base_name=?, f_name=?, area_name=?, seat_name=? WHERE seat_id=?
 * - delete(seatId) → DELETE FROM seats WHERE seat_id=?
 * - findById(seatId) → SELECT * FROM seats WHERE seat_id=?
 */
public class MastersDAO {

    // TODO: 後でMySQL接続に差し替え - 以下の List 操作を SQL の INSERT/UPDATE/DELETE に置き換える
    // SeatsDAO.SEATS を共有することで、予約画面と同じデータを参照している
    private static final List<Seat> seats = SeatsDAO.SEATS;

    public List<Seat> getAll() {
        // TODO: 後でMySQL接続に差し替え: SELECT * FROM seats ORDER BY base_name, f_name, area_name, seat_name
        return new ArrayList<>(seats);
    }

    public Seat findById(int seatId) {
        // TODO: 後でMySQL接続に差し替え: SELECT * FROM seats WHERE seat_id = ?
        return seats.stream().filter(s -> s.getSeatId() == seatId).findFirst().orElse(null);
    }

    public void add(Seat seat) {
        // TODO: 後でMySQL接続に差し替え: INSERT INTO seats (base_name, f_name, area_name, seat_name) VALUES (?, ?, ?,
?)
        seat.setSeatId(SeatsDAO.idCounter.getAndIncrement());
        seats.add(seat);
    }

    public boolean update(Seat updated) {
        // TODO: 後でMySQL接続に差し替え: UPDATE seats SET base_name=?, f_name=?, area_name=?, seat_name=? WHERE seat_
id=?
        for (int i = 0; i < seats.size(); i++) {
            if (seats.get(i).getSeatId() == updated.getSeatId()) {
                seats.set(i, updated);
                return true;
            }
        }
        return false;
    }

    public boolean delete(int seatId) {
        // TODO: 後でMySQL接続に差し替え: DELETE FROM seats WHERE seat_id=?
        return seats.removeIf(s -> s.getSeatId() == seatId);
    }
}
```

java/dao/dao/ReservationsDAO.java

```
package dao;

import entity.Reservation;
import java.time.LocalDate;
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.Collectors;

/**
 * 予約データDAO（モック版）
 * TODO: 後でMySQL接続に差し替え
 * - findById() → SELECT * FROM reservations WHERE user_id=? AND cancel_flag=0
 * - findByDate() → SELECT * FROM reservations WHERE reserve_date=? AND cancel_flag=0
 * - add() → INSERT INTO reservations (...)
 * - cancel() → UPDATE reservations SET cancel_flag=1, cancel_date=? WHERE reserve_id=?
 * - getSeatUsage() → SELECT seat_id, user_id FROM reservations WHERE reserve_date=? AND cancel_flag=0 AND seat_id>0
 */
public class ReservationsDAO {

    // TODO: 後でMySQL接続に差し替え - この静的リストをDBに置き換える
    static final List<Reservation> RESERVATIONS = new ArrayList<>();
    static final AtomicInteger idCounter = new AtomicInteger(1);

    static {
        // 今日の日付を基準にモックデータを生成（DBに切り替え後はこのブロックごと削除）
        String today = LocalDate.now().toString();
        String tomorrow = LocalDate.now().plusDays(1).toString();
        String d3 = LocalDate.now().plusDays(3).toString();

        RESERVATIONS.add(new Reservation(idCounter.getAndIncrement(), "user001", "user001", today, "出社", "", 1, 0, null));
        RESERVATIONS.add(new Reservation(idCounter.getAndIncrement(), "user002", "user002", today, "出社", "", 2, 0, null));
        RESERVATIONS.add(new Reservation(idCounter.getAndIncrement(), "user003", "user003", today, "出社", "", 6, 0, null));
        RESERVATIONS.add(new Reservation(idCounter.getAndIncrement(), "user004", "user004", today, "出社", "", 11, 0, null));
        RESERVATIONS.add(new Reservation(idCounter.getAndIncrement(), "user005", "user005", today, "在宅", "在宅勤務", 0, 0, null));
        RESERVATIONS.add(new Reservation(idCounter.getAndIncrement(), "user001", "user001", tomorrow, "出社", "", 3, 0, null));
        RESERVATIONS.add(new Reservation(idCounter.getAndIncrement(), "user002", "user002", tomorrow, "出張", "大阪出張", 0, 0, null));
        RESERVATIONS.add(new Reservation(idCounter.getAndIncrement(), "user003", "user003", d3, "出社", "", 7, 0, null));
        // キャンセル済みサンプル（画面確認用）
        RESERVATIONS.add(new Reservation(idCounter.getAndIncrement(), "user001", "user001", d3, "出社", "", 4, 1, today));
    }

    public List<Reservation> findById(String userId) {
        // TODO: 後でMySQL接続に差し替え: SELECT * FROM reservations WHERE user_id=? AND cancel_flag=0 ORDER BY reserve_date
        return RESERVATIONS.stream()
            .filter(r -> r.getUserId().equals(userId) && r.getCancelFlag() == 0)
            .collect(Collectors.toList());
    }

    public List<Reservation> findTodayByUserId(String userId) {
        // TODO: 後でMySQL接続に差し替え: SELECT * FROM reservations WHERE user_id=? AND reserve_date=CURDATE() AND ca
    }
}
```



```

ancel_flag=0
    String today = LocalDate.now().toString();
    return RESERVATIONS.stream()
        .filter(r -> r.getUserId().equals(userId)
            && r.getReserveDate().equals(today)
            && r.getCancelFlag() == 0)
        .collect(Collectors.toList());
}

public List<Reservation> findByDate(String date) {
    // TODO: 後でMySQL接続に差し替え: SELECT * FROM reservations WHERE reserve_date=? AND cancel_flag=0
    return RESERVATIONS.stream()
        .filter(r -> r.getReserveDate().equals(date) && r.getCancelFlag() == 0)
        .collect(Collectors.toList());
}

/**
 * 指定日のseatId→userId マップを返す (座席ステータス表示用)
 * key=seatId, value=userId
 */
public Map<Integer, String> getSeatUsageForDate(String date) {
    // TODO: 後でMySQL接続に差し替え: SELECT seat_id, user_id FROM reservations WHERE reserve_date=? AND cancel_flag=0 AND seat_id>0
    Map<Integer, String> map = new HashMap<>();
    RESERVATIONS.stream()
        .filter(r -> r.getReserveDate().equals(date)
            && r.getCancelFlag() == 0
            && r.getSeatId() > 0)
        .forEach(r -> map.put(r.getSeatId(), r.getUserId()));
    return map;
}

public void add(Reservation r) {
    // TODO: 後でMySQL接続に差し替え: INSERT INTO reservations (user_id, register_id, reserve_date, gyosaki, memo, seat_id, cancel_flag) VALUES (...)
    r.setReserveId(idCounter.getAndIncrement());
    RESERVATIONS.add(r);
}

public void cancel(int reserveId) {
    // TODO: 後でMySQL接続に差し替え: UPDATE reservations SET cancel_flag=1, cancel_date=CURDATE() WHERE reserve_id=?
    String today = LocalDate.now().toString();
    RESERVATIONS.stream()
        .filter(r -> r.getReserveId() == reserveId)
        .findFirst()
        .ifPresent(r -> {
            r.setCancelFlag(1);
            r.setCancelDate(today);
        });
}

public Reservation findById(int reserveId) {
    // TODO: 後でMySQL接続に差し替え: SELECT * FROM reservations WHERE reserve_id=?
    return RESERVATIONS.stream()
        .filter(r -> r.getReserveId() == reserveId)
        .findFirst().orElse(null);
}
}

```

java/dao/dao/SeatsDAO.java

```
package dao;

import entity.Seat;
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.Collectors;

/**
 * 座席データアクセスオブジェクト（モック版）
 * TODO: 後でMySQL接続に差し替え
 * - getAllSeats()      → SELECT * FROM seats
 * - findById(id)       → SELECT * FROM seats WHERE seat_id = ?
 * - findByBase(base)   → SELECT * FROM seats WHERE base_name = ?
 * - findByFloor()      → SELECT * FROM seats WHERE base_name=? AND f_name=?
 * - findByArea()       → SELECT * FROM seats WHERE base_name=? AND f_name=? AND area_name=?
 */
public class SeatsDAO {

    // TODO: 後でMySQL接続に差し替え - この静的リストをDBクエリ結果に置き換える
    static final List<Seat> SEATS = new ArrayList<>(Arrays.asList(
        // 三田オフィス 1F Aエリア (seatId 1-5)
        new Seat(1, "三田オフィス", "1F", "Aエリア", "A-01"),
        new Seat(2, "三田オフィス", "1F", "Aエリア", "A-02"),
        new Seat(3, "三田オフィス", "1F", "Aエリア", "A-03"),
        new Seat(4, "三田オフィス", "1F", "Aエリア", "A-04"),
        new Seat(5, "三田オフィス", "1F", "Aエリア", "A-05"),
        // 三田オフィス 1F Bエリア (seatId 6-10)
        new Seat(6, "三田オフィス", "1F", "Bエリア", "B-01"),
        new Seat(7, "三田オフィス", "1F", "Bエリア", "B-02"),
        new Seat(8, "三田オフィス", "1F", "Bエリア", "B-03"),
        new Seat(9, "三田オフィス", "1F", "Bエリア", "B-04"),
        new Seat(10, "三田オフィス", "1F", "Bエリア", "B-05"),
        // 三田オフィス 2F Aエリア (seatId 11-15)
        new Seat(11, "三田オフィス", "2F", "Aエリア", "A-01"),
        new Seat(12, "三田オフィス", "2F", "Aエリア", "A-02"),
        new Seat(13, "三田オフィス", "2F", "Aエリア", "A-03"),
        new Seat(14, "三田オフィス", "2F", "Aエリア", "A-04"),
        new Seat(15, "三田オフィス", "2F", "Aエリア", "A-05"),
        // 三田オフィス 2F Bエリア (seatId 16-20)
        new Seat(16, "三田オフィス", "2F", "Bエリア", "B-01"),
        new Seat(17, "三田オフィス", "2F", "Bエリア", "B-02"),
        new Seat(18, "三田オフィス", "2F", "Bエリア", "B-03"),
        new Seat(19, "三田オフィス", "2F", "Bエリア", "B-04"),
        new Seat(20, "三田オフィス", "2F", "Bエリア", "B-05"),
        // 芝浦オフィス 1F Aエリア (seatId 21-25)
        new Seat(21, "芝浦オフィス", "1F", "Aエリア", "A-01"),
        new Seat(22, "芝浦オフィス", "1F", "Aエリア", "A-02"),
        new Seat(23, "芝浦オフィス", "1F", "Aエリア", "A-03"),
        new Seat(24, "芝浦オフィス", "1F", "Aエリア", "A-04"),
        new Seat(25, "芝浦オフィス", "1F", "Aエリア", "A-05"),
        // 芝浦オフィス 2F Aエリア (seatId 26-30)
        new Seat(26, "芝浦オフィス", "2F", "Aエリア", "A-01"),
        new Seat(27, "芝浦オフィス", "2F", "Aエリア", "A-02"),
        new Seat(28, "芝浦オフィス", "2F", "Aエリア", "A-03"),
        new Seat(29, "芝浦オフィス", "2F", "Aエリア", "A-04"),
        new Seat(30, "芝浦オフィス", "2F", "Aエリア", "A-05")

    ));

    // — 今後オフィスを追加したいときは —————
    // ① ここに「new Seat(座席ID, "ooオフィス", "フロア名", "エリア名", "座席名")」の形式で
    // 座席データを追加する（座席IDは既存と重複しない番号にする）
    // ② OFFICE_IMAGES（このクラスの下の方）に「ooオフィス → 画像ファイル名」の対応を追加する
}
```

```

// ③ WebContent/images (Eclipse側) / src/main/webapp/images (オリジナル側) の両方に
//   フロアマップ画像 (PNG) を追加する
//   ※ かつて存在した「田町オフィス」はデータと画像を削除済み。
//   上の手順を逆に行えば、同じ要領でオフィスを削除できる。
));

// IDカウンター (新規追加時に使用)
static final AtomicInteger idCounter = new AtomicInteger(31);

/**
 * 拠点 (オフィス) 名 → フロアマップ画像ファイル名の対応表。
 * 座席利用状況確認画面で、選んだ拠点に応じた画像を切り替えて表示するために使う。
 * 新しいオフィスを追加したときは、ここにも対応を追記すること。
 */
static final Map<String, String> OFFICE_IMAGES = new LinkedHashMap<>();
static {
    OFFICE_IMAGES.put("三田オフィス", "office_mita.png");
    OFFICE_IMAGES.put("芝浦オフィス", "office_shibaura.png");
}

/** 拠点名に対応するフロアマップ画像ファイル名を返す (未登録の拠点はnull) */
public String getOfficeImage(String baseName) {
    return OFFICE_IMAGES.get(baseName);
}

public List<Seat> getAllSeats() {
    // TODO: 後でMySQL接続に差し替え: SELECT * FROM seats ORDER BY seat_id
    return new ArrayList<>(SEATS);
}

public Seat findById(int seatId) {
    // TODO: 後でMySQL接続に差し替え: SELECT * FROM seats WHERE seat_id = ?
    return SEATS.stream().filter(s -> s.getSeatId() == seatId).findFirst().orElse(null);
}

public List<Seat> findByBase(String baseName) {
    // TODO: 後でMySQL接続に差し替え: SELECT * FROM seats WHERE base_name = ?
    return SEATS.stream()
        .filter(s -> s.getBaseName().equals(baseName))
        .collect(Collectors.toList());
}

public List<Seat> findByFloor(String baseName, String fName) {
    // TODO: 後でMySQL接続に差し替え: SELECT * FROM seats WHERE base_name=? AND f_name=?
    return SEATS.stream()
        .filter(s -> s.getBaseName().equals(baseName) && s.getFName().equals(fName))
        .collect(Collectors.toList());
}

public List<Seat> findByArea(String baseName, String fName, String areaName) {
    // TODO: 後でMySQL接続に差し替え: SELECT * FROM seats WHERE base_name=? AND f_name=? AND area_name=?
    return SEATS.stream()
        .filter(s -> s.getBaseName().equals(baseName)
            && s.getFName().equals(fName)
            && s.getAreaName().equals(areaName))
        .collect(Collectors.toList());
}

/** 絞り込み検索 (nullや空文字は条件なし扱い) */
public List<Seat> filterSeats(String baseName, String fName, String areaName) {
    return SEATS.stream()
        .filter(s -> (baseName == null || baseName.isEmpty() || s.getBaseName().equals(baseName)))
        .filter(s -> (fName == null || fName.isEmpty() || s.getFName().equals(fName)))

```

```

        .filter(s -> (areaName == null || areaName.isEmpty() || s.getAreaName().equals(areaName)))
        .collect(Collectors.toList());
    }

    /** 拠点一覧（重複除去） */
    public List<String> getBases() {
        return SEATS.stream().map(Seat::getBaseName).distinct().collect(Collectors.toList());
    }

    /** 指定拠点のフロア一覧 */
    public List<String> getFloors(String baseName) {
        return SEATS.stream()
            .filter(s -> baseName == null || baseName.isEmpty() || s.getBaseName().equals(baseName))
            .map(Seat::getFName).distinct().collect(Collectors.toList());
    }

    /** 指定拠点・フロアのエリア一覧 */
    public List<String> getAreas(String baseName, String fName) {
        return SEATS.stream()
            .filter(s -> (baseName == null || baseName.isEmpty() || s.getBaseName().equals(baseName)))
            .filter(s -> (fName == null || fName.isEmpty() || s.getFName().equals(fName)))
            .map(Seat::getAreaName).distinct().collect(Collectors.toList());
    }
}

```

3. model（業務ロジック：MVC2のModel。Logicインターフェースを実装）

java/model/model/Logic.java

```

package model;

import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

/**
 * 業務ロジックの共通インターフェース（MVC2の「Model」側の入口）。
 *
 * 画面の流れはどの機能でも共通でこうなっている：
 * ① ブラウザ → ControlServlet (action=oo で振り分け)
 * ② ControlServlet → このインターフェースを実装したLogicクラスの execute()
 * ③ Logic → DAO を呼び出してデータを取得・更新し、req.setAttribute() でJSPに値を渡す
 * ④ Logic → 次に表示するJSPのパスを return し、ControlServlet がそこへ forward する
 *
 * execute() は処理を行い、次に表示するJSPのパスを返す。
 * ControlServlet は返されたパスへ forward する。
 * 新しい機能を追加するときは、この Logic を実装したクラスを1つ作り、
 * ControlServlet の dispatch() に case を1行追加すればよい。
 */
public interface Logic {
    /**
     * @return forward先のJSPパス ("/WEB-INF/jsp/...") 。
     *      null の場合、ControlServlet は forward しない (Logic内でredirect済みの場合) 。
     */
    String execute(HttpServletRequest req, HttpServletResponse res) throws Exception;
}

```

java/model/model/LoginLogic.java

```
package model;

import dao.AccountsDAO;
import dao.ReservationsDAO;
import entity.Account;
import entity.Reservation;
import java.io.IOException;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.util.List;

/**
 * ログイン処理 (action=login で画面表示／action=doLogin で認証) 。
 *
 * ★ここで session にセットする userId / userName / isAdmin / bId の4つは、
 * ログイン後すべての画面・Logicから session.getAttribute(...) で参照される
 * 「誰がログインしているか」の基本情報。新しい画面を作るときもこの4つを使えばよい。
 */
public class LoginLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        String action = req.getParameter("action");

        if ("doLogin".equals(action)) {
            return doLogin(req, res);
        }
        // action=login: ログイン画面を表示するだけ
        return "/WEB-INF/jsp/login.jsp";
    }

    private String doLogin(HttpServletRequest req, HttpServletResponse res) throws IOException {
        String userId = req.getParameter("userId");
        String password = req.getParameter("password");

        Account account = new AccountsDAO().authenticate(userId, password);
        if (account == null) {
            req.setAttribute("errorMsg", "ユーザーIDまたはパスワードが正しくありません。");
            return "/WEB-INF/jsp/loginNG.jsp";
        }

        // セッションを新規発行してユーザー情報を格納
        HttpSession session = req.getSession(true);
        session.setAttribute("userId", account.getUserId());
        session.setAttribute("userName", account.getName());
        session.setAttribute("isAdmin", account.getAdmin() == 1);
        session.setAttribute("bId", account.getBId());

        // 本日の予約を取得してセッションに保持
        List<Reservation> todayRes = new ReservationsDAO().findTodayByUserId(account.getUserId());
        session.setAttribute("todayReservations", todayRes);

        // メニューJSPへ直接forwardすると、ControlServlet#showMenu()が用意する
        // 「同じ部署の出社状況」「本日の予約」「今週の予約状況」などの表示用データが
        // セットされないまま画面が表示されてしまう。必ず action=menu / adminMenu を
        // 経由させて showMenu() を実行させるため、リダイレクトする。
        if (account.getAdmin() == 1) {
            res.sendRedirect(req.getContextPath() + "/ControlServlet?action=adminMenu");
        } else {
            res.sendRedirect(req.getContextPath() + "/ControlServlet?action=menu");
        }
    }
}
```

```
    }  
    return null;  
}  
}
```

java/model/model/LogoutLogic.java

```
package model;  
  
import javax.servlet.http.HttpServletRequest;  
import javax.servlet.http.HttpServletResponse;  
import javax.servlet.http.HttpSession;  
  
/**  
 * ログアウト処理。  
 * セッション（ログイン情報を保持している入れ物）を破棄して、ログアウト完了画面を表示する。  
 * これ以降、ログインし直さない限り action=login 以外のページへはアクセスできない  
 * （ControlServlet.process() のセッションチェックで弾かれる）。  
 */  
public class LogoutLogic implements Logic {  
  
    @Override  
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {  
        // false を渡しているので、セッションが無ければ新規作成しない（無ければ何もしなくてよい）  
        HttpSession session = req.getSession(false);  
        if (session != null) {  
            session.invalidate(); // セッション内の userId / isAdminなどをすべて破棄する  
        }  
        return "/WEB-INF/jsp/logout.jsp";  
    }  
}
```

```

package model;

import dao.AccountsDAO;
import dao.LocationsDAO;
import dao.ReservationsDAO;
import dao.SeatsDAO;
import entity.Account;
import entity.Reservation;
import entity.Seat;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.time.LocalDate;
import java.util.*;

public class ReserveLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        String action = req.getParameter("action");

        if ("doReserve".equals(action)) {
            return doReserve(req);
        }
        // action=reserve: stage で段階表示を切り替える
        return showReserve(req);
    }

    // ----- 座席選択フロー -----

    private String showReserve(HttpServletRequest req) {
        HttpSession session = req.getSession(false);
        SeatsDAO seatsDAO = new SeatsDAO();
        ReservationsDAO resDAO = new ReservationsDAO();
        String today = LocalDate.now().toString();

        String stage = req.getParameter("stage");
        if (stage == null || stage.isEmpty()) stage = "base";

        // パラメータをセッションへ蓄積
        String base = req.getParameter("base");
        String floor = req.getParameter("floor");
        String area = req.getParameter("area");
        String date = req.getParameter("date");

        if (base != null && !base.isEmpty()) session.setAttribute("resBase", base);
        if (floor != null && !floor.isEmpty()) session.setAttribute("resFloor", floor);
        if (area != null && !area.isEmpty()) session.setAttribute("resArea", area);

        // セッションから取得
        String sBase = (String) session.getAttribute("resBase");
        String sFloor = (String) session.getAttribute("resFloor");
        String sArea = (String) session.getAttribute("resArea");

        req.setAttribute("stage", stage);
        req.setAttribute("selBase", sBase);
        req.setAttribute("selFloor", sFloor);
        req.setAttribute("selArea", sArea);

        // 今日の座席利用マップ（埋有率計算用）
    }

```

```

Map<Integer, String> todayUsage = resDAO.getSeatUsageForDate(today);

switch (stage) {
    case "base": {
        // 拠点ごとの埋有率
        // 分母：その拠点の全席数（全フロア・全エリア合計）
        List<Map<String, Object>> items = new ArrayList<>();
        for (String b : seatsDAO.getBases()) {
            List<Seat> baseSeats = seatsDAO.findByBase(b);
            long used = baseSeats.stream().filter(s -> todayUsage.containsKey(s.getSeatId())).count();
            int total = baseSeats.size();
            int rate = total > 0 ? (int)(used * 100 / total) : 0;
            Map<String, Object> m = new HashMap<>();
            m.put("name", b); m.put("total", total); m.put("used", used); m.put("rate", rate);
            items.add(m);
        }
        req.setAttribute("items", items);
        break;
    }
    case "floor": {
        // フロアごとの埋有率
        // 分母：そのフロアの全席数（全エリア合計）
        List<Map<String, Object>> items = new ArrayList<>();
        for (String f : seatsDAO.getFloors(sBase)) {
            List<Seat> floorSeats = seatsDAO.findByFloor(sBase, f);
            long used = floorSeats.stream().filter(s -> todayUsage.containsKey(s.getSeatId())).count();
            int total = floorSeats.size();
            int rate = total > 0 ? (int)(used * 100 / total) : 0;
            Map<String, Object> m = new HashMap<>();
            m.put("name", f); m.put("total", total); m.put("used", used); m.put("rate", rate);
            items.add(m);
        }
        req.setAttribute("items", items);
        break;
    }
    case "area": {
        // エリアごとの埋有率
        // 分母：そのエリアの席のみ
        List<Map<String, Object>> items = new ArrayList<>();
        for (String a : seatsDAO.getAreas(sBase, sFloor)) {
            List<Seat> areaSeats = seatsDAO.findByArea(sBase, sFloor, a);
            long used = areaSeats.stream().filter(s -> todayUsage.containsKey(s.getSeatId())).count();
            int total = areaSeats.size();
            int rate = total > 0 ? (int)(used * 100 / total) : 0;
            Map<String, Object> m = new HashMap<>();
            m.put("name", a); m.put("total", total); m.put("used", used); m.put("rate", rate);
            items.add(m);
        }
        req.setAttribute("items", items);
        break;
    }
    case "seat": {
        // 日付プルダウン（予約できるのは今日から2週間先まで）
        List<String> dates = new ArrayList<>();
        LocalDate d = LocalDate.now();
        for (int i = 0; i < 15; i++) {
            dates.add(d.plusDays(i).toString());
        }
        req.setAttribute("dates", dates);

        // 選択日の座席利用状況
        String selectedDate = (date != null && !date.isEmpty()) ? date : today;
        req.setAttribute("selectedDate", selectedDate);
    }
}

```



```

        session.setAttribute("resDate", selectedDate);

        Map<Integer, String> usage = resDAO.getSeatUsageForDate(selectedDate);
        List<Seat> areaSeats = seatsDAO.findByArea(sBase, sFloor, sArea);

        // 使用中の座席にカーソルを合わせたときに、予約者の氏名・部署名が
        // ひと目で分かるように（座席ボタンには userId しか入っていないため変換する）
        AccountsDAO accDAO = new AccountsDAO();
        LocationsDAO locDAO = new LocationsDAO();
        Map<Integer, String> seatUserNameMap = new HashMap<>();
        Map<Integer, String> seatUserDeptMap = new HashMap<>();
        for (Map.Entry<Integer, String> e : usage.entrySet()) {
            Account acc = accDAO.findById(e.getValue());
            seatUserNameMap.put(e.getKey(), acc != null ? acc.getName() : e.getValue());
            seatUserDeptMap.put(e.getKey(), acc != null ? locDAO.getBNameById(acc.getBId()) : "");
        }

        // エリア埋有率（選択日）
        long usedCount = areaSeats.stream().filter(s -> usage.containsKey(s.getSeatId())).count();
        int total = areaSeats.size();
        int rate = total > 0 ? (int)(usedCount * 100 / total) : 0;

        req.setAttribute("areaSeats", areaSeats);
        req.setAttribute("seatUsage", usage);
        req.setAttribute("seatUserNameMap", seatUserNameMap);
        req.setAttribute("seatUserDeptMap", seatUserDeptMap);
        req.setAttribute("areaRate", rate);
        req.setAttribute("areaUsed", usedCount);
        req.setAttribute("areaTotal", total);
        break;
    }
}
return "/WEB-INF/jsp/reserve.jsp";
}

// ----- 予約実行 -----

private String doReserve(HttpServletRequest req) {
    HttpSession session = req.getSession(false);
    String userId = (String) session.getAttribute("userId");
    String seatIdStr = req.getParameter("seatId");
    String date = req.getParameter("date");
    String memo = req.getParameter("memo");
    if (date == null || date.isEmpty()) date = LocalDate.now().toString();

    if (seatIdStr == null) {
        req.setAttribute("errorMsg", "座席が選択されていません。");
        return "/WEB-INF/jsp/reserveNG.jsp";
    }
    int seatId = Integer.parseInt(seatIdStr);

    // 同一日・同一席に既存予約がないか確認
    ReservationsDAO resDAO = new ReservationsDAO();
    Map<Integer, String> usage = resDAO.getSeatUsageForDate(date);
    if (usage.containsKey(seatId)) {
        req.setAttribute("errorMsg", "選択した座席はすでに予約済みです。");
        return "/WEB-INF/jsp/reserveNG.jsp";
    }

    Reservation r = new Reservation();
    r.setUserId(userId);
    r.setRegisterId(userId);
    r.setReserveDate(date);

```

```
r.setGyosaki("出社");
r.setSeatId(seatId);
r.setMemo(memo != null && !memo.isEmpty() ? memo : null);
r.setCancelFlag(0);
resDAO.add(r);
```

```
// 座席選択の中間値をクリア
```

```
session.removeAttribute("resBase");
session.removeAttribute("resFloor");
session.removeAttribute("resArea");
session.removeAttribute("resDate");
```

```
Seat seat = new SeatsDAO().findById(seatId);
req.setAttribute("reservation", r);
req.setAttribute("seat", seat);
return "/WEB-INF/jsp/reserveOK.jsp";
```

```
}
```

```
}
```

```

package model;

import dao.AccountsDAO;
import dao.LocationsDAO;
import dao.ReservationsDAO;
import dao.SeatsDAO;
import entity.Account;
import entity.Reservation;
import entity.Seat;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.time.LocalDate;
import java.util.*;
import java.util.stream.Collectors;

public class AdminReserveLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        String action = req.getParameter("action");

        if ("doAdminReserve".equals(action)) {
            return doReserve(req);
        }
        return showForm(req);
    }

    private String showForm(HttpServletRequest req) {
        SeatsDAO seatsDAO = new SeatsDAO();
        AccountsDAO accDAO = new AccountsDAO();
        LocationsDAO locDAO = new LocationsDAO();

        // 「〇〇部署の〇〇」形式で検索・表示できるように、社員一覧に部署名を付与
        List<Map<String, Object>> accountRows = new ArrayList<>();
        for (Account a : accDAO.findAll()) {
            Map<String, Object> row = new HashMap<>();
            row.put("userId", a.getUserId());
            row.put("name", a.getName());
            row.put("bName", locDAO.getBNameById(a.getBId()));
            row.put("label", locDAO.getBNameById(a.getBId()) + "の" + a.getName());
            accountRows.add(row);
        }
        req.setAttribute("accountRows", accountRows);
        req.setAttribute("deptNames", locDAO.findAll().stream()
            .map(loc -> loc.getBName()).collect(Collectors.toList()));
        req.setAttribute("bases", seatsDAO.getBases());

        // パラメータ引き継ぎ（段階選択用）
        String base = req.getParameter("base");
        String floor = req.getParameter("floor");
        String area = req.getParameter("area");
        String date = req.getParameter("date");

        if (base != null && !base.isEmpty()) {
            req.setAttribute("selBase", base);
            req.setAttribute("floors", seatsDAO.getFloors(base));
        }
        if (floor != null && !floor.isEmpty() && base != null) {
            req.setAttribute("selFloor", floor);
            req.setAttribute("areas", seatsDAO.getAreas(base, floor));
        }
    }
}

```

```

    }
    if (area != null && !area.isEmpty() && base != null && floor != null) {
        req.setAttribute("selArea", area);
        String selectedDate = (date != null && !date.isEmpty()) ? date : LocalDate.now().toString();
        req.setAttribute("selectedDate", selectedDate);

        // 指定日の座席状況
        ReservationsDAO resDAO = new ReservationsDAO();
        Map<Integer, String> usage = resDAO.getSeatUsageForDate(selectedDate);
        List<Seat> seats = seatsDAO.findByArea(base, floor, area);

        // 使用中の座席にカーソルを合わせたときに、予約者の氏名・部署名が
        // ひと目で分かるように（座席ボタンには userId しか入っていないため変換する）
        Map<Integer, String> seatUserNameMap = new HashMap<>();
        Map<Integer, String> seatUserDeptMap = new HashMap<>();
        for (Map.Entry<Integer, String> e : usage.entrySet()) {
            Account acc = accDAO.findById(e.getValue());
            seatUserNameMap.put(e.getKey(), acc != null ? acc.getName() : e.getValue());
            seatUserDeptMap.put(e.getKey(), acc != null ? locDAO.getBNameById(acc.getBId()) : "");
        }

        req.setAttribute("areaSeats", seats);
        req.setAttribute("seatUsage", usage);
        req.setAttribute("seatUserNameMap", seatUserNameMap);
        req.setAttribute("seatUserDeptMap", seatUserDeptMap);
    }

    // 日付プルダウン（予約できるのは今日から2週間先まで）
    List<String> dates = new ArrayList<>();
    LocalDate d = LocalDate.now();
    for (int i = 0; i < 15; i++) dates.add(d.plusDays(i).toString());
    req.setAttribute("dates", dates);

    return "/WEB-INF/jsp/adminReserve.jsp";
}

private String doReserve(HttpServletRequest req) {
    HttpSession session = req.getSession(false);
    String adminId = (String) session.getAttribute("userId");
    ReservationsDAO resDAO = new ReservationsDAO();
    SeatsDAO seatsDAO = new SeatsDAO();

    String mode    = req.getParameter("mode");    // individual / meeting
    String date    = req.getParameter("date");
    String memo    = req.getParameter("memo");
    String[] seatIds = req.getParameterValues("seatIds"); // 複数選択

    if (date == null || date.isEmpty()) date = LocalDate.now().toString();
    if (seatIds == null || seatIds.length == 0) {
        req.setAttribute("errorMsg", "座席を選択してください。");
        return showForm(req);
    }

    Map<Integer, String> usage = resDAO.getSeatUsageForDate(date);
    List<String> created = new ArrayList<>();

    if ("individual".equals(mode)) {
        // 個人の代理予約（1名・1席）
        if (seatIds.length > 1) {
            req.setAttribute("errorMsg", "個人の代理予約では座席は1席のみ選択してください。");
            return showForm(req);
        }
        String targetUserId = req.getParameter("targetUserId");
    }

```

```

        if (targetUserId == null || targetUserId.isEmpty()) {
            req.setAttribute("errorMsg", "対象社員を選択してください。");
            return showForm(req);
        }
        int seatId = Integer.parseInt(seatIds[0]);
        if (usage.containsKey(seatId)) {
            req.setAttribute("errorMsg", "選択した座席はすでに予約済みです。");
            return showForm(req);
        }
        Reservation r = makeReservation(adminId, targetUserId, date, seatId, memo);
        resDAO.add(r);
        created.add(targetUserId + " → " + getSeatLabel(seatsDAO.findById(seatId)));

    } else {
        // 会議用 (複数名×複数席)
        String[] targetUserIds = req.getParameterValues("targetUserIds");
        if (targetUserIds == null || targetUserIds.length == 0) {
            req.setAttribute("errorMsg", "対象社員を選択してください。");
            return showForm(req);
        }
        if (targetUserIds.length != seatIds.length) {
            req.setAttribute("errorMsg",
                "参加者数 (" + targetUserIds.length + "名)と選択座席数 (" + seatIds.length + "席) が一致しませ
ん。");
            return showForm(req);
        }
        for (int i = 0; i < targetUserIds.length; i++) {
            int seatId = Integer.parseInt(seatIds[i]);
            if (usage.containsKey(seatId)) continue; // すでに使用中はスキップ
            Reservation r = makeReservation(adminId, targetUserIds[i], date, seatId, memo);
            resDAO.add(r);
            created.add(targetUserIds[i] + " → " + getSeatLabel(seatsDAO.findById(seatId)));
        }
    }

    req.setAttribute("createdList", created);
    req.setAttribute("reserveDate", date);
    return "/WEB-INF/jsp/reserveOK.jsp";
}

private Reservation makeReservation(String registerId, String userId, String date, int seatId, String memo) {
    Reservation r = new Reservation();
    r.setRegisterId(registerId); // 予約した管理者
    r.setUserId(userId);         // 実際に座る人
    r.setReserveDate(date);
    r.setGyosaki("出社");
    r.setSeatId(seatId);
    r.setMemo(memo != null && !memo.isEmpty() ? memo : null);
    r.setCancelFlag(0);
    return r;
}

private String getSeatLabel(Seat s) {
    if (s == null) return "不明";
    return s.getBaseName() + " " + s.getFName() + " " + s.getAreaName() + " " + s.getSeatName();
}
}

```

```

package model;

import dao.AccountsDAO;
import dao.ReservationsDAO;
import dao.SeatsDAO;
import entity.Account;
import entity.Reservation;
import entity.Seat;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.util.*;

/**
 * 予約確認・キャンセル（action=reserveConfirm で一覧表示／action=doCancel で取消）。
 * ログイン中の本人の予約一覧を表示し、各行の「取消」ボタンから
 * doCancel を呼ぶと該当予約に取消フラグを立てる（物理削除ではなく論理削除）。
 */
public class ReserveConfirmLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        String action = req.getParameter("action");
        if ("doCancel".equals(action)) {
            return doCancel(req);
        }
        return showConfirm(req);
    }

    private String showConfirm(HttpServletRequest req) {
        HttpSession session = req.getSession(false);
        String userId = (String) session.getAttribute("userId");
        loadReservations(req, userId);
        return "/WEB-INF/jsp/reserveConfirm.jsp";
    }

    private String doCancel(HttpServletRequest req) {
        HttpSession session = req.getSession(false);
        String userId = (String) session.getAttribute("userId");
        String reserveIdStr = req.getParameter("reserveId");

        if (reserveIdStr != null && !reserveIdStr.isEmpty()) {
            int reserveId = Integer.parseInt(reserveIdStr);
            ReservationsDAO resDAO = new ReservationsDAO();
            Reservation r = resDAO.findById(reserveId);
            // 自分の予約のみキャンセル可（セキュリティチェック）
            if (r != null && r.getUserId().equals(userId)) {
                resDAO.cancel(reserveId);
                req.setAttribute("cancelMessage", "予約をキャンセルしました。");
            }
        }
        loadReservations(req, userId);
        return "/WEB-INF/jsp/reserveConfirm.jsp";
    }

    private void loadReservations(HttpServletRequest req, String userId) {
        ReservationsDAO resDAO = new ReservationsDAO();
        SeatsDAO seatsDAO = new SeatsDAO();
        AccountsDAO accDAO = new AccountsDAO();

        List<Reservation> list = resDAO.findById(userId);
    }

```

```
// 座席名マップ (seatId → 座席情報)
Map<Integer, Seat> seatMap = new HashMap<>();
for (Reservation r : list) {
    if (r.getSeatId() > 0 && !seatMap.containsKey(r.getSeatId())) {
        Seat s = seatsDAO.findById(r.getSeatId());
        if (s != null) seatMap.put(r.getSeatId(), s);
    }
}

req.setAttribute("reservations", list);
req.setAttribute("seatMap", seatMap);
}
}
```

java/model/model/LocationRegistLogic.java

```
package model;

import dao.ReservationsDAO;
import entity.Reservation;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.time.LocalDate;

/**
 * 行先登録（出社・在宅・出張のいずれかを今日の行先として登録する）。
 *
 * 「在宅」「出張」はそれぞれ専用のメモ入力画面を経由して登録する
 * （出張先とメモが混在して分かりにくくならないよう、表示時にも区別できる形式で保存する）。
 * 「出社」だけは特別で、ここでは登録せず座席予約画面（ReserveLogic）へリダイレクトし、
 * 座席が決まった時点で「出社＋座席」をまとめて1件登録する
 * （そうしないと「出社（座席なし）」と「出社（座席あり）」の2件ができてしまうため）。
 */
public class LocationRegistLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        String action = req.getParameter("action");

        switch (action) {
            case "locationRegist":
                return "/WEB-INF/jsp/locationRegist.jsp";

            case "doLocationRegist":
                return doRegist(req, res);

            case "businessTrip":
                return "/WEB-INF/jsp/businessTrip.jsp";

            case "doBusinessTrip":
                return doBusinessTrip(req, res);

            case "remoteWork":
                return "/WEB-INF/jsp/remoteWork.jsp";

            case "doRemoteWork":
                return doRemoteWork(req, res);

            default:
                return "/WEB-INF/jsp/locationRegist.jsp";
        }
    }

    private String doRegist(HttpServletRequest req, HttpServletResponse res) throws Exception {
        String gyosaki = req.getParameter("gyosaki"); // 出社 / 在宅 / 出張
        if ("出張".equals(gyosaki)) {
            return "/WEB-INF/jsp/businessTrip.jsp";
        }
        if ("在宅".equals(gyosaki)) {
            return "/WEB-INF/jsp/remoteWork.jsp";
        }
        // 出社：ここでは登録せず座席予約画面（ReserveLogic）へリダイレクトし、
        // 座席が決まった時点で「出社＋座席」をまとめて1件登録する
        // （そうしないと「出社（座席なし）」と「出社（座席あり）」の2件ができてしまうため）
        res.sendRedirect(req.getContextPath() + "/ControlServlet?action=reserve");
        return null;
    }
}
```



```

}

private String doRemoteWork(HttpServletRequest req, HttpServletResponse res) throws Exception {
    String memo = req.getParameter("memo");
    return saveAndGoMenu(req, res, "在宅", memo != null ? memo : "");
}

private String doBusinessTrip(HttpServletRequest req, HttpServletResponse res) throws Exception {
    String destination = req.getParameter("destination");
    String memo = req.getParameter("memo");
    // 出張先とメモを混ぜると後で見たときにどちらか分からなくなるため、
    // 「出張先:oo/メモ:oo」の形式で保存し、表示時にも一目で区別できるようにする
    StringBuilder fullMemo = new StringBuilder("出張先:").append(destination != null ? destination : "");
    if (memo != null && !memo.isEmpty()) {
        fullMemo.append("/メモ:").append(memo);
    }
    return saveAndGoMenu(req, res, "出張", fullMemo.toString());
}

private void saveReservation(HttpServletRequest req, String gyosaki, String memo) {
    HttpSession session = req.getSession(false);
    String userId = (String) session.getAttribute("userId");
    String today = LocalDate.now().toString();

    Reservation r = new Reservation();
    r.setUserId(userId);
    r.setRegisterId(userId);
    r.setReserveDate(today);
    r.setGyosaki(gyosaki);
    r.setMemo(memo);
    r.setSeatId(0);
    r.setCancelFlag(0);

    new ReservationsDAO().add(r);

    // メニューへ戻る前に本日の予約を更新
    session.setAttribute("todayReservations",
        new ReservationsDAO().findTodayByUserId(userId));
}

private String saveAndGoMenu(HttpServletRequest req, HttpServletResponse res, String gyosaki, String memo) throws Exception {
    saveReservation(req, gyosaki, memo);

    HttpSession session = req.getSession(false);
    // リダイレクト後の画面で表示するため、リクエスト属性ではなくセッションに積んでおく
    session.setAttribute("successMsg", gyosaki + "の行先登録が完了しました。");

    boolean isAdmin = Boolean.TRUE.equals(session.getAttribute("isAdmin"));
    res.sendRedirect(req.getContextPath() + "/ControlServlet?action=" + (isAdmin ? "adminMenu" : "menu"));
    return null;
}
}

```

java/model/model/MasterUpdateLogic.java

```
package model;

import dao.MastersDAO;
import dao.SeatsDAO;
import entity.Seat;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.util.List;
import java.util.stream.Collectors;

public class MasterUpdateLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        String action = req.getParameter("action");
        MastersDAO dao = new MastersDAO();

        switch (action) {
            case "masterUpdate":
                // 一覧表示
                setListAttributes(req, dao);
                return "/WEB-INF/jsp/masterUpdate.jsp";

            case "masterEdit": {
                // 更新フォームを表示（対象行を editSeat にセット）
                int id = Integer.parseInt(req.getParameter("seatId"));
                Seat editSeat = dao.findById(id);
                req.setAttribute("editSeat", editSeat);
                setListAttributes(req, dao);
                return "/WEB-INF/jsp/masterUpdate.jsp";
            }

            case "doMasterAdd": {
                // 新規追加
                Seat s = buildSeat(req, 0);
                dao.add(s);
                req.setAttribute("message", "座席「" + s.getSeatName() + "」を追加しました。");
                setListAttributes(req, dao);
                return "/WEB-INF/jsp/masterUpdate.jsp";
            }

            case "doMasterUpdate": {
                // 更新
                int id = Integer.parseInt(req.getParameter("seatId"));
                Seat s = buildSeat(req, id);
                dao.update(s);
                req.setAttribute("message", "座席ID " + id + " を更新しました。");
                setListAttributes(req, dao);
                return "/WEB-INF/jsp/masterUpdate.jsp";
            }

            case "doMasterDelete": {
                // 削除
                int id = Integer.parseInt(req.getParameter("seatId"));
                dao.delete(id);
                req.setAttribute("message", "座席ID " + id + " を削除しました。");
                setListAttributes(req, dao);
                return "/WEB-INF/jsp/masterUpdate.jsp";
            }

            default:

```

```

        setListAttributes(req, dao);
        return "/WEB-INF/jsp/masterUpdate.jsp";
    }
}

private Seat buildSeat(HttpServletRequest req, int seatId) {
    Seat s = new Seat();
    s.setSeatId(seatId);

    // 「既存オフィスから選択」または「新しいオフィス名を入力」のどちらかが baseName に入る。
    // 新規オフィス入力欄 (newBaseName) に値があれば、そちらを優先する。
    // ※オフィスという名前のマスタ行は無く、座席を1件追加した時点でそのオフィスが新しく登場する仕組み。
    String newBase = req.getParameter("newBaseName");
    String baseName = (newBase != null && !newBase.trim().isEmpty())
        ? newBase.trim() : req.getParameter("baseName");

    s.setBaseName(baseName);
    s.setFName(req.getParameter("fName"));
    s.setAreaName(req.getParameter("areaName"));
    s.setSeatName(req.getParameter("seatName"));
    return s;
}

/**
 * 一覧表示・絞り込み・フロアマップ画像表示に必要な属性をまとめてセットする。
 * 「① オフィスを選んで見る」ための bases / selectedBase / officeImage と、
 * 選んだオフィスだけに絞り込んだ座席一覧 seats をリクエストに積む。
 */
private void setListAttributes(HttpServletRequest req, MastersDAO dao) {
    SeatsDAO seatsDAO = new SeatsDAO();
    String selectedBase = req.getParameter("filterBase");

    List<Seat> all = dao.getAll();
    List<Seat> seats = (selectedBase != null && !selectedBase.isEmpty())
        ? all.stream().filter(s -> selectedBase.equals(s.getBaseName())).collect(Collectors.toList())
        : all;

    req.setAttribute("seats", seats);
    req.setAttribute("bases", seatsDAO.getBases());
    req.setAttribute("selectedBase", selectedBase);
    req.setAttribute("officeImage", (selectedBase != null && !selectedBase.isEmpty())
        ? seatsDAO.getOfficeImage(selectedBase) : null);
}
}

```

```

package model;

import dao.AccountsDAO;
import dao.LocationsDAO;
import dao.ReservationsDAO;
import dao.SeatsDAO;
import entity.Account;
import entity.Reservation;
import entity.Seat;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.time.LocalDate;
import java.util.*;

/**
 * 氏名検索（入力されたキーワードを含む氏名のアカウントを探し、各人の所属部署と
 * 本日の出社状況・座席を一覧表示する）。
 *
 * keyword パラメータが無ければ検索フォームだけを表示し、
 * あれば AccountsDAO.findByName() で部分一致検索した結果に
 * 部署名・本日の行先・座席情報を付け加えて JSP に渡す。
 */
public class NameSearchLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        String keyword = req.getParameter("keyword");

        if (keyword != null) {
            // 検索実行
            AccountsDAO accDAO = new AccountsDAO();
            LocationsDAO locDAO = new LocationsDAO();
            ReservationsDAO resDAO = new ReservationsDAO();
            SeatsDAO seatDAO = new SeatsDAO();
            String today = LocalDate.now().toString();
            Map<Integer, String> todayUsage = resDAO.getSeatUsageForDate(today);

            List<Account> results = accDAO.findByName(keyword.trim());

            // 検索結果に各ユーザーの本日のステータスと予約座席を付与
            List<Map<String, Object>> rows = new ArrayList<>();
            for (Account a : results) {
                Map<String, Object> row = new HashMap<>();
                row.put("account", a);
                row.put("bName", locDAO.getBNameById(a.getBId()));

                List<Reservation> todayRes = resDAO.findTodayByUserId(a.getUserId());
                String todayStatus = "未登録";
                Seat seat = null;
                if (!todayRes.isEmpty()) {
                    Reservation r = todayRes.get(0);
                    todayStatus = r.getGyosaki();
                    if (r.getSeatId() > 0) {
                        seat = seatDAO.findById(r.getSeatId());
                    }
                }
                row.put("todayStatus", todayStatus);
                row.put("seat", seat);
                rows.add(row);
            }
            req.setAttribute("keyword", keyword);

```

```
        req.setAttribute("results", rows);
    }
    return "/WEB-INF/jsp/nameSearch.jsp";
}
}
```

```

package model;

import dao.AccountsDAO;
import dao.LocationsDAO;
import dao.ReservationsDAO;
import dao.SeatsDAO;
import entity.Account;
import entity.Location;
import entity.Reservation;
import entity.Seat;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.util.*;

/**
 * 部門検索（指定した部署のメンバー一覧と、各メンバーの本日の出勤状況を表示する）。
 *
 * 流れ：
 * 1) 画面表示時：部署の一覧（プルダウン用）を取得して渡す
 * 2) 部署が選択されたとき（bId/パラメータあり）：
 *     その部署のメンバーを取得し、1人ずつ「本日の予約」を調べて
 *     「氏名」「本日の行先（出社/在宅/出張/未登録）」「座席（出社の場合）」をまとめてJSPに渡す
 *
 * 複数のDAO（Accounts → Reservations → Seats）を組み合わせて1画面分のデータを作る、
 * Logicクラスの典型的な役割が分かりやすい例になっている。
 */
public class DepartmentSearchLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        LocationsDAO locDAO = new LocationsDAO();
        List<Location> departments = locDAO.findAll();
        req.setAttribute("departments", departments); // 検索フォームの部署プルダウン用

        String bIdStr = req.getParameter("bId");
        if (bIdStr != null && !bIdStr.isEmpty()) {
            int bId = Integer.parseInt(bIdStr);
            AccountsDAO accDAO = new AccountsDAO();
            ReservationsDAO resDAO = new ReservationsDAO();
            SeatsDAO seatDAO = new SeatsDAO();

            // 選んだ部署に所属するメンバー全員分を、1人ずつ「本日の状況」付きで組み立てる
            List<Account> members = accDAO.findByBId(bId);
            List<Map<String, Object>> rows = new ArrayList<>();
            for (Account a : members) {
                Map<String, Object> row = new HashMap<>();
                row.put("account", a);

                // 本日の予約を取得し、行先（出社/在宅/出張）と座席（出社の場合のみ）を判定する
                List<Reservation> todayRes = resDAO.findTodayByUserId(a.getUserId());
                String todayStatus = "未登録";
                Seat seat = null;
                if (!todayRes.isEmpty()) {
                    Reservation r = todayRes.get(0);
                    todayStatus = r.getGyosaki();
                    if (r.getSeatId() > 0) {
                        seat = seatDAO.findById(r.getSeatId());
                    }
                }
                row.put("todayStatus", todayStatus);
                row.put("seat", seat);
            }
        }
    }
}

```

```
        rows.add(row);
    }
    req.setAttribute("selectedBId", bId);
    req.setAttribute("selectedBName", locDAO.getBNameById(bId));
    req.setAttribute("results", rows);
}
return "/WEB-INF/jsp/departmentSearch.jsp";
}
```

4. control (フロントコントローラ：すべてのリクエストの入口)

java/control/control/ControlServlet.java

```
package control;

import dao.AccountsDAO;
import dao.ReservationsDAO;
import dao.SeatsDAO;
import entity.Account;
import entity.Reservation;
import entity.Seat;
import model.*;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.io.IOException;
import java.time.LocalDate;
import java.util.*;

/**
 * Za-Navi 統合フロントコントローラ。
 * すべてのリクエストを /ControlServlet?action=oo で受け取り、
 * 対応する Logic を呼び出してから JSP へ forward する。
 */
public class ControlServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        process(req, res);
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        process(req, res);
    }

    private void process(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        req.setCharacterEncoding("UTF-8");
        res.setCharacterEncoding("UTF-8");

        String action = req.getParameter("action");
        if (action == null || action.isEmpty()) action = "login";

        // セッションチェック (ログイン不要アクションを除く)
        if (!action.equals("login") && !action.equals("doLogin")) {
            HttpSession session = req.getSession(false);
            if (session == null || session.getAttribute("userId") == null) {
                try {
                    res.sendRedirect(req.getContextPath() + "/ControlServlet?action=login");
                } catch (IOException e) { /* already committed */ }
                return;
            }
        }

        String jspPath = null;
        try {
```



```

        jspPath = dispatch(action, req, res);
    } catch (Exception e) {
        req.setAttribute("errorMsg", "システムエラー: " + e.getMessage());
        jspPath = "/WEB-INF/jsp/loginNG.jsp";
    }

    if (jspPath != null && !res.isCommitted()) {
        req.getRequestDispatcher(jspPath).forward(req, res);
    }
}

// ----- アクション振り分け -----

private String dispatch(String action, HttpServletRequest req, HttpServletResponse res)
    throws Exception {

    switch (action) {

        // --- ログイン / ログアウト ---
        case "login":
        case "doLogin":
            return new LoginLogic().execute(req, res);

        case "logout":
            return new LogoutLogic().execute(req, res);

        // --- 一般メニュー ---
        case "menu":
            return showMenu(req, false);

        // --- 管理者メニュー ---
        case "adminMenu":
            return showMenu(req, true);

        // --- 行先登録 ---
        case "locationRegist":
        case "doLocationRegist":
        case "businessTrip":
        case "doBusinessTrip":
        case "remoteWork":
        case "doRemoteWork":
            return new LocationRegistLogic().execute(req, res);

        // --- 座席予約 ---
        case "reserve":
        case "doReserve":
            return new ReserveLogic().execute(req, res);

        // --- 予約確認・キャンセル ---
        case "reserveConfirm":
        case "doCancel":
            return new ReserveConfirmLogic().execute(req, res);

        // --- 氏名検索 ---
        case "nameSearch":
        case "doNameSearch":
            return new NameSearchLogic().execute(req, res);

        // --- 部門検索 ---
        case "departmentSearch":
        case "doDepartmentSearch":
            return new DepartmentSearchLogic().execute(req, res);
    }
}

```

```

// — 座席利用状況（管理） —
case "seatStatus":
case "doSeatStatus":
    return showSeatStatus(req);

// — 代理予約（管理） —
case "adminReserve":
case "doAdminReserve":
    return new AdminReserveLogic().execute(req, res);

// — マスタ更新（管理） —
case "masterUpdate":
case "masterEdit":
case "doMasterAdd":
case "doMasterUpdate":
case "doMasterDelete":
    return new MasterUpdateLogic().execute(req, res);

default:
    return "/WEB-INF/jsp/login.jsp";
}
}

// ————— メニュー共通 —————

private String showMenu(HttpServletRequest req, boolean isAdmin) {
    HttpSession session = req.getSession(false);
    String userId = (String) session.getAttribute("userId");

    AccountsDAO accDAO = new AccountsDAO();
    ReservationsDAO resDAO = new ReservationsDAO();

    List<Reservation> todayRes = resDAO.findTodayByUserId(userId);
    session.setAttribute("todayReservations", todayRes);
    req.setAttribute("todayReservations", todayRes);

    // 同じ部署の本日の出社人数・座席一覧（管理者メニューだけに表示する情報なので、管理者のときだけ集計する）
    Account me = accDAO.findById(userId);
    if (isAdmin && me != null) {
        SeatsDAO seatDAO = new SeatsDAO();
        List<Account> deptMembers = accDAO.findByBId(me.getBId());

        List<Map<String, Object>> deptSeatList = new ArrayList<>();
        long officeCount = 0;
        for (Account a : deptMembers) {
            List<Reservation> mRes = resDAO.findTodayByUserId(a.getUserId());

            // 本日の状況（未登録の場合は「未登録」のままにする）
            String todayStatus = "未登録";
            Seat seat = null;
            if (!mRes.isEmpty()) {
                Reservation r = mRes.get(0);
                todayStatus = r.getGyosaki();
                if ("出社".equals(todayStatus)) {
                    officeCount++;
                    seat = r.getSeatId() > 0 ? seatDAO.findById(r.getSeatId()) : null;
                }
            }

            Map<String, Object> row = new HashMap<>();
            row.put("name", a.getName());
            row.put("todayStatus", todayStatus);
            row.put("seat", seat);

```

```

        deptSeatList.add(row);
    }
    req.setAttribute("deptMemberCount", deptMembers.size());
    req.setAttribute("deptOfficeCount", officeCount);
    req.setAttribute("deptSeatList", deptSeatList);
}

// 管理者向け：今日から2週間分の予約状況サマリ（人数だけでなく、誰がどの席も見られるように）
if (isAdmin) {
    SeatsDAO seatDAO = new SeatsDAO();
    List<Map<String, Object>> twoWeekSummary = new ArrayList<>();
    for (int i = 0; i < 14; i++) {
        String date = LocalDate.now().plusDays(i).toString();
        List<Reservation> dayRes = resDAO.findByDate(date);
        long officeNum = dayRes.stream().filter(r -> "出社".equals(r.getGyosaki())).count();
        long seatNum = dayRes.stream().filter(r -> r.getSeatId() > 0).count();

        // 出社・座席が確定している人の「氏名+座席」明細（折りたたみ表示用）
        List<Map<String, Object>> people = new ArrayList<>();
        for (Reservation r : dayRes) {
            if (!"出社".equals(r.getGyosaki()) || r.getSeatId() <= 0) continue;
            Account acc = accDAO.findById(r.getUserId());
            Seat seat = seatDAO.findById(r.getSeatId());
            Map<String, Object> p = new HashMap<>();
            p.put("name", acc != null ? acc.getName() : r.getUserId());
            p.put("seat", seat);
            people.add(p);
        }

        Map<String, Object> row = new HashMap<>();
        row.put("date", date);
        row.put("officeCount", officeNum);
        row.put("seatCount", seatNum);
        row.put("people", people);
        twoWeekSummary.add(row);
    }
    req.setAttribute("twoWeekSummary", twoWeekSummary);
}

return isAdmin ? "/WEB-INF/jsp/adminMenu.jsp" : "/WEB-INF/jsp/menu.jsp";
}

// ----- 座席利用状況 -----

private String showSeatStatus(HttpServletRequest req) {
    SeatsDAO seatsDAO = new SeatsDAO();
    ReservationsDAO resDAO = new ReservationsDAO();
    AccountsDAO accDAO = new AccountsDAO();
    dao.LocationsDAO locDAO = new dao.LocationsDAO();

    String filterBase = req.getParameter("filterBase");
    String filterFloor = req.getParameter("filterFloor");
    String filterArea = req.getParameter("filterArea");
    String filterDate = req.getParameter("filterDate");
    if (filterDate == null || filterDate.isEmpty()) filterDate = LocalDate.now().toString();

    // 座席利用状況の確認は「今日を基準に2週間前～2週間後」までに限定する
    // （カレンダーで自由に選べてしまうと際限なく過去・未来を確認できてしまうため）
    LocalDate today = LocalDate.now();
    LocalDate minDate = today.minusDays(14);
    LocalDate maxDate = today.plusDays(14);
    LocalDate parsedDate;
    try {

```

```

        parsedDate = LocalDate.parse(filterDate);
    } catch (Exception e) {
        parsedDate = today;
    }
    if (parsedDate.isBefore(minDate)) parsedDate = minDate;
    if (parsedDate.isAfter(maxDate)) parsedDate = maxDate;
    filterDate = parsedDate.toString();
    req.setAttribute("minDate", minDate.toString());
    req.setAttribute("maxDate", maxDate.toString());

    // 日付はカレンダーUIだと押しづらいとの声があったため、他の項目と同じ
    // セレクトボックス形式（2週間前～2週間後を一覧）で選べるようにする
    String[] youbi = {"日", "月", "火", "水", "木", "金", "土"};
    LinkedHashMap<String, String> dateOptions = new LinkedHashMap<>();
    for (LocalDate d = minDate; !d.isAfter(maxDate); d = d.plusDays(1)) {
        String label = d.getMonthValue() + "/" + d.getDayOfMonth()
            + " (" + youbi[d.getDayOfWeek().getValue() % 7] + ") "
            + (d.equals(today) ? " ※本日" : "");
        dateOptions.put(d.toString(), label);
    }
    req.setAttribute("dateOptions", dateOptions);

    // ドロップダウン用リスト（「すべて」は用意せず、拠点→フロア→エリアの順に選んでもらう）
    req.setAttribute("bases", seatsDAO.getBases());
    boolean baseChosen = filterBase != null && !filterBase.isEmpty();
    boolean floorChosen = baseChosen && filterFloor != null && !filterFloor.isEmpty();
    boolean areaChosen = floorChosen && filterArea != null && !filterArea.isEmpty();

    if (baseChosen) req.setAttribute("floors", seatsDAO.getFloors(filterBase));
    if (floorChosen) req.setAttribute("areas", seatsDAO.getAreas(filterBase, filterFloor));

    // 拠点・フロア・エリアがすべて決まったときだけ座席を表示する
    // （「〇〇オフィスの〇〇エリア」のように、常に具体的な場所が一意に決まる状態にするため）
    List<Seat> filteredSeats = areaChosen
        ? seatsDAO.filterSeats(filterBase, filterFloor, filterArea)
        : new ArrayList<>();

    // 指定日の利用状況
    Map<Integer, String> seatUsageMap = resDAO.getSeatUsageForDate(filterDate);

    // 使用中ユーザーの「氏名」「部署名」マップ（座席マップ上で氏名の横に部署名も出すため）
    Map<Integer, String> seatUserNameMap = new HashMap<>();
    Map<Integer, String> seatUserDeptMap = new HashMap<>();
    for (Map.Entry<Integer, String> e : seatUsageMap.entrySet()) {
        entity.Account acc = accDAO.findById(e.getValue());
        seatUserNameMap.put(e.getKey(), acc != null ? acc.getName() : e.getValue());
        seatUserDeptMap.put(e.getKey(), acc != null ? locDAO.getBNameById(acc.getBId()) : "");
    }

    long usedCount = filteredSeats.stream()
        .filter(s -> seatUsageMap.containsKey(s.getSeatId()))
        .count();
    long freeCount = filteredSeats.size() - usedCount;

    // ログイン中の管理者と同じ部署で、本日「出社」する人の一覧（氏名・座席）
    // ※座席利用状況の確認画面で「自分の部署のメンバーが今日どこに座っているか」をすぐ把握できるようにするため
    HttpSession statusSession = req.getSession(false);
    String myUserId = (String) statusSession.getAttribute("userId");
    Account me = accDAO.findById(myUserId);
    List<Map<String, Object>> deptTodayList = new ArrayList<>();
    if (me != null) {
        for (Account a : accDAO.findByBId(me.getBId())) {
            List<Reservation> mRes = resDAO.findTodayByUserId(a.getUserId());
            if (mRes.isEmpty() || !"出社".equals(mRes.get(0).getGyosaki())) continue;

```

```

        Reservation r = mRes.get(0);
        Seat seat = r.getSeatId() > 0 ? seatsDAO.findById(r.getSeatId()) : null;
        Map<String, Object> row = new HashMap<>();
        row.put("name", a.getName());
        row.put("seat", seat);
        deptTodayList.add(row);
    }
}

req.setAttribute("filteredSeats", filteredSeats);
req.setAttribute("seatUsageMap", seatUsageMap);
req.setAttribute("seatUserNameMap", seatUserNameMap);
req.setAttribute("seatUserDeptMap", seatUserDeptMap);
req.setAttribute("usedCount", usedCount);
req.setAttribute("freeCount", freeCount);
req.setAttribute("filterBase", filterBase);
req.setAttribute("filterFloor", filterFloor);
req.setAttribute("filterArea", filterArea);
req.setAttribute("filterDate", filterDate);
req.setAttribute("myDeptName", me != null ? locDAO.getBNameById(me.getBId()) : null);
req.setAttribute("deptTodayList", deptTodayList);

return "/WEB-INF/jsp/seatStatus.jsp";
}
}

```

5. JSP共通部品（ヘッダー：ログイン中ユーザー表示・メニュー導線）

webapp/WEB-INF/jsp/common/WEB-INF/jsp/common/header.jspf

```

<%@ page pageEncoding="UTF-8" %>
<!-- 一般ユーザー用ヘッダー（常にteal系の配色のまま。管理者がここに来た場合も「一般ユーザーとして見ている」と
分かるように、わざと配色を切り替えない。管理者用の配色変更は headerAdmin.jspf 側だけで行う） -->
<header class="header">
    <div class="logo">座・Navi <span>座席管理システム</span></div>
    <div class="user-info">
        <details class="menu-dropdown">
            <summary>
                
                <%= session.getAttribute("userName") %> さん
                <span class="caret">&#9662;</span>
            </summary>
            <div class="dropdown-panel">
                <a href="<%= request.getContextPath() %>/ControlServlet?action=menu">メニュー</a>
                <% if (Boolean.TRUE.equals(session.getAttribute("isAdmin"))) { %>
                <a href="<%= request.getContextPath() %>/ControlServlet?action=adminMenu">管理メニューへ戻る</a>
                <% } %>
                <a href="<%= request.getContextPath() %>/ControlServlet?action=logout">ログアウト</a>
            </div>
        </details>
    </div>
</header>

```

```

<%@ page pageEncoding="UTF-8" %>
<!-- 管理者用ヘッダー -->
<style>
  /* 管理者画面は常にインディゴ/ネイビー系の基調色にする（一般ユーザー画面のティール系と区別するため）。
     style.css の :root で定義された --teal 系の変数をここで上書きするだけで、
     ヘッダー・ボタン・バッジなど teal 変数を使っている箇所がまとめて変わる。 */
  :root { --teal:#3949AB; --teal-light:#7986CB; --teal-dark:#283593; }
</style>
<header class="header">
  <div class="logo">座・Navi <span>座席管理システム</span><span class="admin-badge">管理者</span></div>
  <div class="user-info">
    <details class="menu-dropdown">
      <summary>
        
        <%= session.getAttribute("userName") %> さん
        <span class="caret">&#9662;</span>
      </summary>
      <div class="dropdown-panel">
        <a href="<%= request.getContextPath() %>/ControlServlet?action=adminMenu">管理メニュー</a>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=menu">一般メニュー</a>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=logout">ログアウト</a>
      </div>
    </details>
  </div>
</header>

```

6. JSP（ログイン・メニュー画面）

webapp/WEB-INF/jsp/WEB-INF/jsp/login.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>ログイン - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<div class="login-page">
  <div class="login-card">
    <div class="login-title">座・Navi</div>
    <div class="login-subtitle">座席管理システム</div>

    <% if (request.getAttribute("errorMsg") != null) { %>
    <div class="alert alert-danger"><%= request.getAttribute("errorMsg") %></div>
    <% } %>

    <form action="<%= request.getContextPath() %>/ControlServlet" method="post">
      <input type="hidden" name="action" value="doLogin">

      <div class="form-group">
        <label>ユーザーID</label>
        <input type="text" name="userId" placeholder="例: user001" required autofocus>
      </div>
      <div class="form-group">
        <label>パスワード</label>
        <input type="password" name="password" placeholder="パスワードを入力" required>
      </div>

      <div style="display:flex; gap:10px; margin-top:24px;">
        <button type="submit" class="btn btn-primary btn-lg" style="flex:1;">
          ログイン
        </button>
      </div>
    </form>

    <div style="margin-top:20px; padding-top:16px; border-top:1px solid #ECEFF1; text-align:center;">
      <p style="font-size:12px; color:#757575; margin-bottom:10px;">管理者の方はこちら</p>
      <form action="<%= request.getContextPath() %>/ControlServlet" method="post">
        <input type="hidden" name="action" value="doLogin">
        <div class="form-group" style="margin-bottom:8px;">
          <input type="text" name="userId" placeholder="管理者ID（例: admin001）" required>
        </div>
        <div class="form-group" style="margin-bottom:12px;">
          <input type="password" name="password" placeholder="パスワード" required>
        </div>
        <button type="submit" class="btn btn-outline" style="width:100%;">
          管理者ログイン
        </button>
      </form>
    </div>

    <!-- ▼▼▼ 差し替え注意: ここは「動作確認用のテストID/パスワードを画面に表示している」サンプル表示です。
    本番環境（実際のデータベースに繋いだ後）では、ログイン情報を画面に表示するのはセキュリティ上NGなので、
    この <div> ごと削除すること。 ▼▼▼ -->
    <div style="margin-top:16px; font-size:11px; color:#aaa; text-align:center;">
```

```
※ デモ: user001/pass001 ・ admin001/admin001
</div>
<!-- ▲▲▲ 差し替え注意ここまで ▲▲▲ -->
</div>
</div>
</body>
</html>
```

webapp/WEB-INF/jsp/WEB-INF/jsp/loginNG.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>ログイン失敗 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<div class="login-page">
  <div class="login-card">
    <div class="login-title">座・Navi</div>
    <div class="alert alert-danger" style="margin:16px 0;">
      <%= request.getAttribute("errorMsg") != null ? request.getAttribute("errorMsg") : "ログインに失敗しました。" %>
    </div>
    <a href="<%= request.getContextPath() %>/ControlServlet?action=login" class="btn btn-primary" style="width:100%; justify-content:center;">
      ログイン画面に戻る
    </a>
  </div>
</div>
</body>
</html>
```

webapp/WEB-INF/jsp/WEB-INF/jsp/logout.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>ログアウト - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<div class="login-page">
  <div class="login-card" style="text-align:center;">
    <div class="login-title">座・Navi</div>
    <div style="margin:24px 0; font-size:48px;">Ⓜ️</div>
    <p style="font-size:16px; font-weight:600; margin-bottom:8px;">ログアウトしました</p>
    <p style="font-size:13px; color:#757575; margin-bottom:28px;">ご利用ありがとうございました。</p>
    <a href="<%= request.getContextPath() %>/ControlServlet?action=login" class="btn btn-primary btn-lg" style="width:100%; justify-content:center;">
      ログイン画面へ
    </a>
  </div>
</div>
</body>
</html>
```



```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.List, entity.Reservation, dao.SeatsDAO, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>メニュー - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<div class="container">

    <%
        // 行先登録などリダイレクト後に表示するメッセージはセッションに積まれているため、
        // 表示したら一度だけ出るように消しておく（フラッシュメッセージ）
        Object successMsg = session.getAttribute("successMsg");
        if (successMsg != null) session.removeAttribute("successMsg");
    %>

    <% if (successMsg != null) { %>
    <div class="alert alert-success"><%= successMsg %></div>
    <% } %>

    <!-- 本日の予約 -->
    <div class="card">
        <h2>本日の予約状況</h2>
        <%
            List<Reservation> todayRes = (List<Reservation>) session.getAttribute("todayReservations");
            if (todayRes == null) todayRes = new java.util.ArrayList<>();
            SeatsDAO seatsDAO = new SeatsDAO();
        %>

        <% if (todayRes.isEmpty()) { %>
        <p style="color:#757575;">本日の予約はまだありません。</p>
        <% } else { %>
        <table class="table">
            <thead>
                <tr><th>日付</th><th>行先</th><th>座席</th><th>メモ</th></tr>
            </thead>
            <tbody>
                <% for (Reservation r : todayRes) {
                    Seat seat = (r.getSeatId() > 0) ? seatsDAO.findById(r.getSeatId()) : null;
                    String seatLabel = seat != null
                        ? seat.getBaseName() + " " + seat.getFName() + " " + seat.getSeatName()
                        : "-";
                %>
                <tr>
                    <td><%= r.getReserveDate() %></td>
                    <td><span class="badge badge-green"><%= r.getGyosaki() %></span></td>
                    <td><%= seatLabel %></td>
                    <td><%= r.getMemo() != null ? r.getMemo() : "" %></td>
                </tr>
                <% } %>
            </tbody>
        </table>
        <% } %>
    </div>

    <!-- 同じ部署の出社状況 -->
    <%
        @SuppressWarnings("unchecked")
    %>

```

```

List<java.util.Map<String, Object>> deptSeatList =
    (List<java.util.Map<String, Object>>) request.getAttribute("deptSeatList");
%>
<% if (request.getAttribute("deptMemberCount") != null) { %>
<div class="card">
    <h2>同じ部署の本日の出社状況</h2>
    <div class="summary-row">
        <span class="badge badge-green">本日出社 : <%= request.getAttribute("deptOfficeCount") %> 名</span>
        <span class="badge badge-gray">部署人数 : <%= request.getAttribute("deptMemberCount") %> 名</span>
    </div>
    <% if (deptSeatList != null && !deptSeatList.isEmpty()) { %>
    <table class="table" style="margin-top:12px;">
        <thead><tr><th>氏名</th><th>本日の状況</th><th>座席</th></tr></thead>
        <tbody>
            <% for (java.util.Map<String, Object> row : deptSeatList) {
                Seat seat = (Seat) row.get("seat");
                String seatLabel = seat != null
                    ? seat.getBaseName() + " " + seat.getFName() + " " + seat.getAreaName() + " " + seat.getSeatName()
                    : "-";
            %>
            <tr>
                <td><%= row.get("name") %></td>
                <td><span class="badge badge-green"><%= row.get("todayStatus") %></span></td>
                <td><%= seatLabel %></td>
            </tr>
            <% } %>
        </tbody>
    </table>
    <% } %>
</div>
<% } %>

<!-- 機能メニュー -->
<div class="card">
    <h2>機能メニュー</h2>
    <div class="menu-grid">
        <a href="<%= request.getContextPath() %>/ControlServlet?action=locationRegist" class="menu-btn">
            行先登録
        </a>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=reserve" class="menu-btn">
            座席予約
        </a>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=reserveConfirm" class="menu-btn">
            予約確認
        </a>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=nameSearch" class="menu-btn">
            氏名検索
        </a>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=departmentSearch" class="menu-btn">
            部門検索
        </a>
    </div>
</div>
</div>
</body>
</html>

```

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.List, java.util.Map, entity.Reservation, dao.SeatsDAO, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>管理者メニュー - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/headerAdmin.jspf" %>

<div class="container">

    <%
        // 行先登録などリダイレクト後に表示するメッセージはセッションに積まれているため、
        // 表示したら一度だけ出るように消しておく（フラッシュメッセージ）
        Object successMsg = session.getAttribute("successMsg");
        if (successMsg != null) session.removeAttribute("successMsg");
    %>

    <% if (successMsg != null) { %>
    <div class="alert alert-success"><%= successMsg %></div>
    <% } %>

    <!-- 本日の予約 -->
    <div class="card">
        <h2>本日の予約状況（管理者ビュー）</h2>
        <%
            List<Reservation> todayRes = (List<Reservation>) session.getAttribute("todayReservations");
            if (todayRes == null) todayRes = new java.util.ArrayList<>();
            SeatsDAO seatsDAO = new SeatsDAO();
        %>

        <% if (todayRes.isEmpty()) { %>
        <p style="color:#757575;">本日の予約はありません。</p>
        <% } else { %>
        <table class="table">
            <thead><tr><th>日付</th><th>行先</th><th>座席</th><th>メモ</th></tr></thead>
            <tbody>
            <% for (Reservation r : todayRes) {
                Seat seat = r.getSeatId() > 0 ? seatsDAO.findById(r.getSeatId()) : null;
                String seatLabel = seat != null ? seat.getBaseName()+" "+seat.getFName()+" "+seat.getSeatName() : "-";
            %>
            <tr>
                <td><%= r.getReserveDate() %></td>
                <td><span class="badge badge-green"><%= r.getGyosaki() %></span></td>
                <td><%= seatLabel %></td>
                <td><%= r.getMemo() != null ? r.getMemo() : "" %></td>
            </tr>
            <% } %>
            </tbody>
        </table>
        <% } %>
    </div>

    <!-- 同じ部署の出社状況 -->
    <%
        @SuppressWarnings("unchecked")
        List<Map<String, Object>> deptSeatList =
            (List<Map<String, Object>>) request.getAttribute("deptSeatList");
    %>

    <% if (request.getAttribute("deptMemberCount") != null) { %>

```

```

<div class="card">
  <h2>同じ部署の本日の出社状況</h2>
  <div class="summary-row">
    <span class="badge badge-green">本日出社：<%= request.getAttribute("deptOfficeCount") %> 名</span>
    <span class="badge badge-gray">部署人数：<%= request.getAttribute("deptMemberCount") %> 名</span>
  </div>
  <% if (deptSeatList != null && !deptSeatList.isEmpty()) { %>
  <div class="summary-row" style="margin-top:10px; gap:8px;">
    <% for (Map<String, Object> row : deptSeatList) {
      Seat seat = (Seat) row.get("seat");
      String seatLabel = seat != null
        ? seat.getFName() + " " + seat.getAreaName() + " " + seat.getSeatName()
        : "座席未定";

      %>
      <span class="badge badge-gray"><%= row.get("name") %> <span style="color:#00695C;">(<%= seatLabel %>)</span>
    </span>
    <% } %>
  </div>
  <% } %>
</div>
<% } %>

<!-- 2週間分の予約状況（管理者のみ）：行をクリックすると氏名・座席の詳細が開く -->
<%
  List<Map<String, Object>> twoWeekSummary = (List<Map<String, Object>>) request.getAttribute("twoWeekSummary");
  %>
  <% if (twoWeekSummary != null) { %>
  <div class="card">
    <h2>今後2週間の予約状況</h2>
    <p style="font-size:12px; color:#757575; margin-bottom:8px;">日付をクリックすると、その日に出社する人の氏名と座席が確認できます。</p>
    <% for (Map<String, Object> row : twoWeekSummary) {
      @SuppressWarnings("unchecked")
      List<Map<String, Object>> people = (List<Map<String, Object>>) row.get("people");
      %>
      <details class="day-summary">
        <summary>
          <span class="day-date"><%= row.get("date") %></span>
          <span class="badge badge-green"><%= row.get("officeCount") %> 名出社</span>
          <span class="badge badge-gray"><%= row.get("seatCount") %> 席予約済み</span>
          <span class="caret">&#9662;</span>
        </summary>
        <div class="day-detail">
          <% if (people == null || people.isEmpty()) { %>
          <p style="color:#9E9E9E; font-size:13px; margin:0;">座席が確定している予約はまだありません。</p>
          <% } else { %>
          <div class="summary-row" style="gap:8px;">
            <% for (Map<String, Object> p : people) {
              Seat seat = (Seat) p.get("seat");
              String seatLabel = seat != null
                ? seat.getBaseName() + " " + seat.getFName() + " " + seat.getAreaName() + " " + seat.getSeatName()
                : "—";

              %>
              <span class="badge badge-gray"><%= p.get("name") %> <span style="color:#00695C;">(<%= seatLabel %>)</span>
            </span>
            <% } %>
          </div>
          <% } %>
        </div>
      </details>
    <% } %>
  </div>
</div>

```

```

<% } %>

<!-- 一般機能 -->
<div class="card">
  <h2>一般機能</h2>
  <div class="menu-grid">
    <a href="<%= request.getContextPath() %>/ControlServlet?action=locationRegist" class="menu-btn">
      行先登録
    </a>
    <a href="<%= request.getContextPath() %>/ControlServlet?action=reserve" class="menu-btn">
      座席予約
    </a>
    <a href="<%= request.getContextPath() %>/ControlServlet?action=reserveConfirm" class="menu-btn">
      予約確認
    </a>
    <a href="<%= request.getContextPath() %>/ControlServlet?action=nameSearch" class="menu-btn">
      氏名検索
    </a>
    <a href="<%= request.getContextPath() %>/ControlServlet?action=departmentSearch" class="menu-btn">
      部門検索
    </a>
  </div>
</div>

<!-- 管理機能 -->
<div class="card">
  <h2>管理機能</h2>
  <div class="menu-grid">
    <a href="<%= request.getContextPath() %>/ControlServlet?action=seatStatus" class="menu-btn">
      座席利用状
    </a>
    <a href="<%= request.getContextPath() %>/ControlServlet?action=adminReserve" class="menu-btn">
      代理予約
    </a>
    <a href="<%= request.getContextPath() %>/ControlServlet?action=masterUpdate" class="menu-btn">
      マスタ更新
    </a>
  </div>
</div>
</div>
</body>
</html>

```

7. JSP（座席予約・行先登録の画面）

webapp/WEB-INF/jsp/WEB-INF/jsp/reserve.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.*, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>座席予約 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>
<%
    String stage    = (String) request.getAttribute("stage");
    if (stage == null) stage = "base";
    String selBase   = (String) request.getAttribute("selBase");
    String selFloor  = (String) request.getAttribute("selFloor");
    String selArea   = (String) request.getAttribute("selArea");
    String ctx       = request.getContextPath();
    String menuAction = Boolean.TRUE.equals(session.getAttribute("isAdmin")) ? "adminMenu" : "menu";

    // stage → 番号マッピング（ステージバー用）
    int stageNum = "base".equals(stage) ? 1 : "floor".equals(stage) ? 2 : "area".equals(stage) ? 3 : 4;
%>

<div class="container">
    <div class="card">
        <h2>座席予約</h2>

        <!-- ステージバー -->
        <div class="stage-bar">
            <div class="stage-step" <%= stageNum >= 1 ? (stageNum > 1 ? "done" : "active") : "" %>>① 拠点</div>
            <div class="stage-step" <%= stageNum >= 2 ? (stageNum > 2 ? "done" : "active") : "" %>>② フロア</div>
            <div class="stage-step" <%= stageNum >= 3 ? (stageNum > 3 ? "done" : "active") : "" %>>③ エリア</div>
            <div class="stage-step" <%= stageNum >= 4 ? "active" : "" %>>④ 座席</div>
        </div>

        <!-- パンくず -->
        <div class="breadcrumb">
            <a href="<%= ctx %>/ControlServlet?action=reserve">拠点選択</a>
            <% if (selBase != null) { %>
                <span class="sep">&rsquo;</span>
                <a href="<%= ctx %>/ControlServlet?action=reserve&stage=floor&base=<%= java.net.URLEncoder.encode(selBase, "UTF-8") %>"><%= selBase %></a>
            <% } %>
            <% if (selFloor != null && selBase != null) { %>
                <span class="sep">&rsquo;</span>
                <a href="<%= ctx %>/ControlServlet?action=reserve&stage=area&base=<%= java.net.URLEncoder.encode(selBase, "UTF-8") %>">&floor=<%= java.net.URLEncoder.encode(selFloor, "UTF-8") %>"><%= selFloor %></a>
            <% } %>
            <% if (selArea != null && selFloor != null) { %>
                <span class="sep">&rsquo;</span>
                <span><%= selArea %></span>
            <% } %>
        </div>

        <!-- ===== 拠点ステージ ===== -->
        <% if ("base".equals(stage)) { %>
            <p style="color:#757575; margin-bottom:16px;">予約したい拠点を選んでください。</p>
        <% } %>
    </div>
</div>
```

```

<div class="occ-grid">
<%
    List<Map<String, Object>> items = (List<Map<String, Object>>) request.getAttribute("items");
    if (items != null) for (Map<String, Object> item : items) {
        String name = (String) item.get("name");
        int rate = (Integer) item.get("rate");
        int total = (Integer) item.get("total");
        Object usedObj = item.get("used");
        long used = usedObj instanceof Long ? (Long)usedObj : ((Integer)usedObj).longValue();
        String barClass = rate < 50 ? "" : rate < 80 ? "med" : rate < 100 ? "high" : "full";
        String encoded = java.net.URLEncoder.encode(name, "UTF-8");
    }
%>
    <a class="occ-card"
        href="<%= ctx %>/ControlServlet?action=reserve&stage=floor&base=<%= encoded %>">
        <div class="occ-name"><%= name %></div>
        <div class="occ-bar-bg">
            <div class="occ-bar <%= barClass %>" style="width:<%= rate %>%;"></div>
        </div>
        <div class="occ-pct"><%= rate %><span style="font-size:14px;">%</span></div>
        <!-- 分母：その拠点の全席（全フロア・全エリア合計）が分母 -->
        <div class="occ-label"><%= used %> / <%= total %> 席 使用中</div>
    </a>
<% } %>
</div>

<!-- ===== フロアステージ ===== -->
<% } else if ("floor".equals(stage)) { %>
    <p style="color:#757575; margin-bottom:16px;">フロアを選んでください。</p>
    <div class="occ-grid">
    <%
        List<Map<String, Object>> items = (List<Map<String, Object>>) request.getAttribute("items");
        if (items != null) for (Map<String, Object> item : items) {
            String name = (String) item.get("name");
            int rate = (Integer) item.get("rate");
            int total = (Integer) item.get("total");
            Object usedObj = item.get("used");
            long used = usedObj instanceof Long ? (Long)usedObj : ((Integer)usedObj).longValue();
            String barClass = rate < 50 ? "" : rate < 80 ? "med" : rate < 100 ? "high" : "full";
            String encodedBase = java.net.URLEncoder.encode(selBase, "UTF-8");
            String encodedFloor = java.net.URLEncoder.encode(name, "UTF-8");
        }
    %>
        <a class="occ-card"
            href="<%= ctx %>/ControlServlet?action=reserve&stage=area&base=<%= encodedBase %>&floor=<%= encodedFloor %>">
            <div class="occ-name"><%= name %></div>
            <div class="occ-bar-bg">
                <div class="occ-bar <%= barClass %>" style="width:<%= rate %>%;"></div>
            </div>
            <div class="occ-pct"><%= rate %><span style="font-size:14px;">%</span></div>
            <!-- 分母：そのフロアの全席（全エリア合計）が分母 -->
            <div class="occ-label"><%= used %> / <%= total %> 席 使用中</div>
        </a>
    <% } %>
    </div>

    <!-- ===== エリアステージ ===== -->
    <% } else if ("area".equals(stage)) { %>
        <p style="color:#757575; margin-bottom:16px;">エリアを選んでください。</p>
        <div class="occ-grid">
        <%
            List<Map<String, Object>> items = (List<Map<String, Object>>) request.getAttribute("items");
            if (items != null) for (Map<String, Object> item : items) {
                String name = (String) item.get("name");
            }
        %>

```



```

<% String barClass2 = areaRate < 50 ? "" : areaRate < 80 ? "med" : areaRate < 100 ? "high" : "full"; %>
<div style="display:inline-block; min-width:220px;">
    <div style="font-size:13px; font-weight:700; color:#757575; margin-bottom:6px;">
        <%= selectedDate %> の埋有率 (<%= selArea %>)
    </div>
    <div class="occ-bar-bg">
        <div class="occ-bar <%= barClass2 %>" style="width:<%= areaRate %>%;"></div>
    </div>
    <!-- 分母：そのエリアの席のみ -->
    <div style="margin-top:4px; font-size:13px;">
        <span class="occ-pct" style="font-size:20px;"><%= areaRate %>%</span>
        <span class="occ-label"> (<%= areaUsed %> / <%= areaTotal %> 席使用中) </span>
    </div>
</div>
</div>

<!-- ③ オフィスレイアウト画像（プレースホルダー） -->
<div class="layout-placeholder">
    オフィスレイアウト画像<br> (images フォルダに画像を追加して差し替え)
</div>

<!-- ④ 座席ボタン（緑=空き、赤=使用中） -->
<p style="font-size:13px; font-weight:700; margin-bottom:8px;">座席を選択してください</p>
<div class="summary-row">
    <span class="badge badge-green">■ 空き席</span>
    <span class="badge badge-red">■ 使用中</span>
</div>

<!-- 空き席ボタンを押すと、下のメモを添えて予約する -->
<form method="post" action="<%= ctx %>/ControlServlet">
    <input type="hidden" name="action" value="doReserve">
    <input type="hidden" name="date" value="<%= selectedDate %>">

    <div class="form-group">
        <label for="memo">メモ（任意）</label>
        <textarea id="memo" name="memo" rows="2" placeholder="例：午後から外出予定 など" style="width:100%; max-width:480px;"></textarea>
    </div>

    <div class="seat-grid">
        <% if (areaSeats != null) for (Seat s : areaSeats) {
            boolean occupied = usage != null && usage.containsKey(s.getSeatId());
            String uname = (occupied && userNameMap != null && userNameMap.containsKey(s.getSeatId()))
                ? userNameMap.get(s.getSeatId()) : "";
            String udept = (occupied && userDeptMap != null && userDeptMap.containsKey(s.getSeatId()))
                ? userDeptMap.get(s.getSeatId()) : "";
            String hoverInfo = (udept != null && !udept.isEmpty()) ? (udept + " " + uname) : uname;
        %>
        <% if (!occupied) { %>
            <!-- 緑ボタン：押すと予約完了 -->
            <button type="submit" name="seatId" value="<%= s.getSeatId() %>" class="seat-btn available">
                <span class="seat-id"><%= s.getSeatName() %></span>
                <span class="seat-user">空き</span>
            </button>
        <% } else { %>
            <!-- 赤ボタン：使用中（クリック不可）。誰が使っているか分かるよう、
                氏名（部署名）を表示しつつ、カーソルを合わせると title 属性で
                「部署名 氏名」がツールチップ表示される -->
            <button type="button" class="seat-btn occupied" disabled title="<%= hoverInfo %>">
                <span class="seat-id"><%= s.getSeatName() %></span>
                <span class="seat-user"><%= uname %><% if (udept != null && !udept.isEmpty()) { %><span style="font-size:11px; opacity:.8;"> (<%= udept %>) </span><% } %></span>
            </button>
        <% } %>
    </div>
</form>

```

```

        <% } %>
    <% } %>
</div>
</form>

<script>
    // 日付プルダウンを変更すると画面が再読み込みされてメモが消えてしまうため、
    // 切り替え前の入力内容を一時保存しておき、再読み込み後に書き戻す。
    // ※ select.form.submit() はブラウザ標準のメソッド呼び出しで "submit" イベントが
    // 発火しないため、addEventListener('submit', ...) では保存できない。
    // そのため、保存してから送信する関数を onchange から直接呼び出す形にしている。
    var RESERVE_MEMO_KEY = 'reserveMemoDraft';

    function submitReserveDateForm(sel) {
        var memo = document.getElementById('memo');
        if (memo) sessionStorage.setItem(RESERVE_MEMO_KEY, memo.value);
        sel.form.submit();
    }

    (function() {
        var memo = document.getElementById('memo');
        if (!memo) return;
        var saved = sessionStorage.getItem(RESERVE_MEMO_KEY);
        if (saved !== null) {
            memo.value = saved;
            sessionStorage.removeItem(RESERVE_MEMO_KEY);
        }
    })();
</script>

<% } %>

<div style="margin-top:24px;">
    <a href="<%= ctx %>/ControlServlet?action=<%= menuAction %>" class="btn btn-secondary">&larr; メニューへ戻る</a
>
</div>
</div>
</div>
</body>
</html>

```

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.List, entity.Reservation, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>予約完了 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<div class="container">
  <div class="card" style="max-width:600px; margin:0 auto; text-align:center;">
    <div style="font-size:56px; margin-bottom:16px;">&#10003;</div>
    <h2 style="color:var(--green); border:none; text-align:center; margin-bottom:20px;">予約が完了しました</h2>

    <%
      // 通常の単席予約
      Reservation r = (Reservation) request.getAttribute("reservation");
      Seat seat = (Seat) request.getAttribute("seat");
      // 代理予約（複数）
      List<String> createdList = (List<String>) request.getAttribute("createdList");
      String reserveDate = (String) request.getAttribute("reserveDate");
      String menuAction = Boolean.TRUE.equals(session.getAttribute("isAdmin")) ? "adminMenu" : "menu";
    %>

    <% if (r != null && seat != null) { %>
    <table class="table" style="text-align:left; max-width:400px; margin:0 auto 24px;">
      <tr><th style="width:40%;">予約日</th><td><%= r.getReserveDate() %></td></tr>
      <tr><th>拠点</th> <td><%= seat.getBaseName() %></td></tr>
      <tr><th>フロア</th><td><%= seat.getFName() %></td></tr>
      <tr><th>エリア</th><td><%= seat.getAreaName() %></td></tr>
      <tr><th>座席</th> <td><strong><%= seat.getSeatName() %></strong></td></tr>
      <% if (r.getMemo() != null && !r.getMemo().isEmpty()) { %>
      <tr><th>メモ</th> <td><%= r.getMemo() %></td></tr>
      <% } %>
    </table>
    <% } %>

    <% if (createdList != null && !createdList.isEmpty()) { %>
    <p style="font-weight:700; margin-bottom:10px;">代理予約完了 (<%= reserveDate %>) </p>
    <ul style="list-style:none; text-align:left; display:inline-block; margin-bottom:20px;">
      <% for (String line : createdList) { %>
      <li style="padding:4px 0; border-bottom:1px solid #eee;">&#10003; <%= line %></li>
      <% } %>
    </ul>
    <% } %>

    <div style="display:flex; gap:12px; justify-content:center;">
      <a href="<%= request.getContextPath() %>/ControlServlet?action=reserveConfirm" class="btn btn-primary">
        予約一覧を確認する
      </a>
      <a href="<%= request.getContextPath() %>/ControlServlet?action=<%= menuAction %>" class="btn btn-secondary">
        メニューへ戻る
      </a>
    </div>
  </div>
</div>

```

```
</body>
</html>
```

webapp/WEB-INF/jsp/WEB-INF/jsp/reserveNG.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>予約失敗 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<%
    String menuAction = Boolean.TRUE.equals(session.getAttribute("isAdmin")) ? "adminMenu" : "menu";
%>

<div class="container">
    <div class="card" style="max-width:500px; margin:0 auto; text-align:center;">
        <div style="font-size:56px; color:var(--red); margin-bottom:16px;">&#10007;</div>
        <h2 style="color:var(--red); border:none; text-align:center; margin-bottom:16px;">予約できませんでした</h2>
        <div class="alert alert-danger">
            <%= request.getAttribute("errorMsg") != null ? request.getAttribute("errorMsg") : "予約処理中にエラーが発生しました。" %>
        </div>
        <div style="display:flex; gap:12px; justify-content:center; margin-top:16px;">
            <a href="<%= request.getContextPath() %>/ControlServlet?action=reserve" class="btn btn-primary">
                座席選択に戻る
            </a>
            <a href="<%= request.getContextPath() %>/ControlServlet?action=<%= menuAction %>" class="btn btn-secondary">
                メニューへ
            </a>
        </div>
    </div>
</div>
</body>
</html>
```

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.*, entity.Reservation, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>予約確認 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<div class="container">
  <div class="card">
    <h2>予約確認・キャンセル</h2>

    <!-- キャンセル完了メッセージ -->
    <% if (request.getAttribute("cancelMessage") != null) { %>
    <div class="alert alert-success" style="font-size:15px; font-weight:600;">
      &#10003; <%= request.getAttribute("cancelMessage") %>
    </div>
    <% } %>

    <%
    List<Reservation> reservations = (List<Reservation>) request.getAttribute("reservations");
    Map<Integer, Seat> seatMap = (Map<Integer, Seat>) request.getAttribute("seatMap");
    String ctx = request.getContextPath();
    String menuAction = Boolean.TRUE.equals(session.getAttribute("isAdmin")) ? "adminMenu" : "menu";
    %>

    <% if (reservations == null || reservations.isEmpty()) { %>
    <p style="color:#757575;">有効な予約はありません。</p>
    <% } else { %>
    <table class="table">
      <thead>
        <tr>
          <th>予約ID</th><th>予約日</th><th>行先</th><th>座席</th><th>メモ</th><th>操作</th>
        </tr>
      </thead>
      <tbody>
        <% for (Reservation r : reservations) {
          Seat seat = r.getSeatId() > 0 && seatMap != null ? seatMap.get(r.getSeatId()) : null;
          String seatLabel = seat != null
            ? seat.getBaseName() + " " + seat.getFName() + " " + seat.getSeatName()
            : "-";
          %>
          <tr>
            <td><%= r.getReserveId() %></td>
            <td><%= r.getReserveDate() %></td>
            <td><span class="badge badge-green"><%= r.getGyosaki() %></span></td>
            <td><%= seatLabel %></td>
            <td><%= r.getMemo() != null ? r.getMemo() : "" %></td>
            <td>
              <!-- キャンセルボタン：POSTで送信（論理削除） -->
              <button type="submit" form="cancelForm<%= r.getReserveId() %>" class="btn btn-danger btn-sm">
                キャンセル
              </button>
            </td>
          </tr>
        <% } %>
      </tbody>
    </table>
  </div>

```

```
</table>

<!-- キャンセル用フォーム（テーブル外に配置） -->
<% for (Reservation r : reservations) { %>
<form id="cancelForm<%= r.getReserveId() %>" action="<%= ctx %>/ControlServlet" method="post" style="display:n
one;">
    <input type="hidden" name="action" value="doCancel">
    <input type="hidden" name="reserveId" value="<%= r.getReserveId() %>">
</form>
<% } %>
<% } %>

<div style="margin-top:20px;">
    <a href="<%= ctx %>/ControlServlet?action=<%= menuAction %>" class="btn btn-secondary">&larr; メニューへ戻る</a
>
</div>
</div>
</body>
</html>
```

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>行先登録 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<%
    String menuAction = Boolean.TRUE.equals(session.getAttribute("isAdmin")) ? "adminMenu" : "menu";
%>

<div class="container">
    <div class="card">
        <h2>行先登録</h2>
        <p style="color:#757575; margin-bottom:24px;">本日の行先を選択してください。</p>

        <div class="gyosaki-group">
            <!-- 出社 -->
            <form action="<%= request.getContextPath() %>/ControlServlet" method="post" style="flex:1; min-width:120px;">
                <input type="hidden" name="action" value="doLocationRegist">
                <input type="hidden" name="gyosaki" value="出社">
                <button type="submit" class="gyosaki-btn office" style="width:100%;">
                    
                    出社
                </button>
            </form>

            <!-- 在宅（次の画面で任意のメモを入力） -->
            <form action="<%= request.getContextPath() %>/ControlServlet" method="post" style="flex:1; min-width:120px;">
                <input type="hidden" name="action" value="doLocationRegist">
                <input type="hidden" name="gyosaki" value="在宅">
                <button type="submit" class="gyosaki-btn remote" style="width:100%;">
                    
                    在宅
                </button>
            </form>

            <!-- 出張 -->
            <form action="<%= request.getContextPath() %>/ControlServlet" method="post" style="flex:1; min-width:120px;">
                <input type="hidden" name="action" value="doLocationRegist">
                <input type="hidden" name="gyosaki" value="出張">
                <button type="submit" class="gyosaki-btn trip" style="width:100%;">
                    
                    出張
                </button>
            </form>
        </div>

        <div style="margin-top:24px;">
            <a href="<%= request.getContextPath() %>/ControlServlet?action=<%= menuAction %>" class="btn btn-secondary">
                &larr; メニューへ戻る
            </a>
        </div>
    </div>
</div>

```

```
</div>
</body>
</html>
```

webapp/WEB-INF/jsp/WEB-INF/jsp/remoteWork.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>在宅メモ入力 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<div class="container">
  <div class="card" style="max-width:500px; margin:0 auto;">
    <h2>在宅勤務として登録します</h2>
    <p style="color:#757575; margin-bottom:24px;">必要であれば、業務内容などのメモを残せます（任意）。</p>

    <form action="<%= request.getContextPath() %>/ControlServlet" method="post">
      <input type="hidden" name="action" value="doRemoteWork">

      <div class="form-group">
        <label>メモ（任意）</label>
        <textarea name="memo" rows="2" placeholder="例：午後から商談対応のため在宅 など" style="width:100%;"></textarea>
      </div>

      <div style="display:flex; gap:10px; margin-top:8px;">
        <button type="submit" class="btn btn-primary" style="flex:1;">
          登録する
        </button>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=locationRegist" class="btn btn-secondary">
          戻る
        </a>
      </div>
    </form>
  </div>
</div>
</body>
</html>
```



```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>出張先入力 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<div class="container">
  <div class="card" style="max-width:500px; margin:0 auto;">
    <h2>出張先を入力してください</h2>

    <form action="<%= request.getContextPath() %>/ControlServlet" method="post">
      <input type="hidden" name="action" value="doBusinessTrip">

      <div class="form-group">
        <label>出張先</label>
        <input type="text" name="destination" placeholder="例: 大阪支社" required autofocus>
      </div>

      <div class="form-group">
        <label>メモ（任意）</label>
        <textarea name="memo" rows="2" placeholder="例：午後から客先訪問のため帰社は夕方 など" style="width:100%;"></tex
tarea>
      </div>

      <div style="display:flex; gap:10px; margin-top:8px;">
        <button type="submit" class="btn btn-primary" style="flex:1;">
          登録する
        </button>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=locationRegist" class="btn btn-secondary">
          戻る
        </a>
      </div>
    </form>
  </div>
</div>
</body>
</html>
```

8. JSP（管理者機能の画面：代理予約・マスタ更新・座席利用状況確認）

webapp/WEB-INF/jsp/WEB-INF/jsp/adminReserve.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.*, entity.Account, entity.Seat, dao.SeatsDAO" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>代理予約 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
<style>
    .mode-card { border:2px solid #CFD8DC; border-radius:8px; padding:16px; cursor:pointer; transition:.15s; }
    .mode-card.active { border-color:var(--teal); background:#E0F2F1; }
    .step-section { margin-top:20px; padding-top:20px; border-top:1px solid #ECEFF1; }
</style>
</head>
<body>
<%@ include file="common/headerAdmin.jspf" %>

<div class="container">
    <div class="card">
        <h2>代理予約</h2>

        <% if (request.getAttribute("errorMsg") != null) { %>
        <div class="alert alert-danger"><%= request.getAttribute("errorMsg") %></div>
        <% } %>

        <%
        @SuppressWarnings("unchecked")
        List<Map<String,Object>> accountRows = (List<Map<String,Object>>) request.getAttribute("accountRows");
        List<String> bases      = (List<String>) request.getAttribute("bases");
        List<String> floors     = (List<String>) request.getAttribute("floors");
        List<String> areas      = (List<String>) request.getAttribute("areas");
        List<String> dates      = (List<String>) request.getAttribute("dates");
        List<Seat> areaSeats    = (List<Seat>) request.getAttribute("areaSeats");
        Map<Integer,String> seatUsage = (Map<Integer,String>) request.getAttribute("seatUsage");
        Map<Integer,String> seatUserNameMap = (Map<Integer,String>) request.getAttribute("seatUserNameMap");
        Map<Integer,String> seatUserDeptMap = (Map<Integer,String>) request.getAttribute("seatUserDeptMap");
        String selBase      = (String) request.getAttribute("selBase");
        String selFloor     = (String) request.getAttribute("selFloor");
        String selArea      = (String) request.getAttribute("selArea");
        String selDate      = (String) request.getAttribute("selectedDate");
        String ctx = request.getContextPath();
        %>

        <!-- 予約タイプ選択 -->
        <p style="font-weight:700; margin-bottom:12px;">⑧ 予約タイプを選択</p>
        <div style="display:flex; gap:14px; margin-bottom:20px;">
            <div class="mode-card" id="modeCardIndividual" onclick="setMode('individual')" style="flex:1;">
                <strong>個人の代理予約（1席）</strong>
                <p style="font-size:12px; color:#757575; margin-top:6px;">対象社員1名を選び、1席を予約します</p>
            </div>
            <div class="mode-card" id="modeCardMeeting" onclick="setMode('meeting')" style="flex:1;">
                <strong>会議用（複数席まとめて予約）</strong>
                <p style="font-size:12px; color:#757575; margin-top:6px;">複数の参加者と座席をまとめて予約します</p>
            </div>
        </div>

        <form action="<%= ctx %>/ControlServlet" method="post" id="adminReserveForm">
            <input type="hidden" name="action" value="doAdminReserve">
```

```

<input type="hidden" name="mode" value="individual" id="modeInput">

<%
  @SuppressWarnings("unchecked")
  List<String> deptNames = (List<String>) request.getAttribute("deptNames");
%>

<!-- ② 対象社員を検索して追加（modeによって切り替え） -->
<div class="step-section" id="sectionIndividual">
  <p style="font-weight:700; margin-bottom:10px;">② 対象社員を検索して追加（個人・1名）</p>
  <div class="filter-row">
    <div class="form-group">
      <label>部署で絞り込み（任意）</label>
      <select id="searchDeptIndividual">
        <option value="">すべての部署</option>
        <% if (deptNames != null) for (String d : deptNames) { %>
          <option value="<%= d %%"><%= d %%"</option>
        <% } %>
      </select>
    </div>
    <div class="form-group">
      <label>氏名で検索（任意）</label>
      <input type="text" id="searchNameIndividual" placeholder="例： 山田">
    </div>
    <div>
      <button type="button" class="btn btn-outline" onclick="searchAccounts('individual')>検索</button>
    </div>
  </div>
  <div id="searchResultIndividual" class="check-group" style="margin-top:10px;">
    <p style="color:#9E9E9E; font-size:13px;">部署または氏名で検索すると、ここに候補が表示されます。</p>
  </div>

  <p style="font-weight:700; margin:18px 0 8px;">選択中の対象社員</p>
  <div id="selectedIndividual" class="check-group">
    <p style="color:#9E9E9E; font-size:13px;">まだ誰も追加されていません。検索して「追加」を押してください。</p>
  </div>
  <div id="hiddenIndividual"></div>
</div>

<div class="step-section" id="sectionMeeting" style="display:none;">
  <p style="font-weight:700; margin-bottom:10px;">② 参加者を検索して追加（複数可）</p>
  <div class="filter-row">
    <div class="form-group">
      <label>部署で絞り込み（任意）</label>
      <select id="searchDeptMeeting">
        <option value="">すべての部署</option>
        <% if (deptNames != null) for (String d : deptNames) { %>
          <option value="<%= d %%"><%= d %%"</option>
        <% } %>
      </select>
    </div>
    <div class="form-group">
      <label>氏名で検索（任意）</label>
      <input type="text" id="searchNameMeeting" placeholder="例： 佐藤">
    </div>
    <div>
      <button type="button" class="btn btn-outline" onclick="searchAccounts('meeting')>検索</button>
    </div>
  </div>
  <div id="searchResultMeeting" class="check-group" style="margin-top:10px;">
    <p style="color:#9E9E9E; font-size:13px;">部署または氏名で検索すると、ここに候補が表示されます。</p>
  </div>

```

```

<p style="font-weight:700; margin:18px 0 8px 0 8px;">選択中の参加者</p>
<div id="selectedMeeting" class="check-group">
  <p style="color:#9E9E9E; font-size:13px;">まだ誰も追加されていません。検索して「追加」を押してください。</p>
</div>
<div id="hiddenMeeting"></div>
</div>

```

```

<!-- ③ 日付・拠点・フロア・エリア選択 -->

```

```

<div class="step-section">
  <p style="font-weight:700; margin-bottom:10px;">③ 日付と場所を選択</p>
  <div class="filter-row">
    <div class="form-group">
      <label>予約日</label>
      <select name="date">
        <% if (dates != null) for (String d : dates) { %>
          <option value="<%= d %%" <%= d.equals(selDate) ? "selected" : "" %%"><%= d %%"</option>
        <% } %>
      </select>
    </div>
    <div class="form-group">
      <label>拠点</label>
      <select name="base" onchange="reloadWith('base', this.value)">
        <option value="">-- 選択 --</option>
        <% if (bases != null) for (String b : bases) { %>
          <option value="<%= b %%" <%= b.equals(selBase) ? "selected" : "" %%"><%= b %%"</option>
        <% } %>
      </select>
    </div>
    <div class="form-group">
      <label>フロア</label>
      <select name="floor" onchange="reloadWith('floor', this.value)">
        <option value="">-- 選択 --</option>
        <% if (floors != null) for (String f : floors) { %>
          <option value="<%= f %%" <%= f.equals(selFloor) ? "selected" : "" %%"><%= f %%"</option>
        <% } %>
      </select>
    </div>
    <div class="form-group">
      <label>エリア</label>
      <select name="area" onchange="reloadWith('area', this.value)">
        <option value="">-- 選択 --</option>
        <% if (areas != null) for (String a : areas) { %>
          <option value="<%= a %%" <%= a.equals(selArea) ? "selected" : "" %%"><%= a %%"</option>
        <% } %>
      </select>
    </div>
  </div>

```

```

<!-- 拠点を選べると、そのオフィスのフロアマップ画像を表示する -->
<%
  String officeImage = (selBase != null && !selBase.isEmpty())
    ? new SeatsDAO().getOfficeImage(selBase) : null;
%>
<% if (officeImage != null) { %>
  <div style="margin-top:14px;">
    のフロアマップ"
      style="max-width:100%; border:1px solid #ECEFF1; border-radius:8px;">
  </div>
  <% } %>
</div>

```

```

<!-- ④ 座席選択 -->

```

```

<% if (areaSeats != null && !areaSeats.isEmpty()) { %>

```

```

<div class="step-section">
  <p style="font-weight:700; margin-bottom:10px;">㊤ 座席を選択してください（個人は1席、会議用は複数選択可）</p>
  <div class="summary-row">
    <span class="badge badge-green">■ 空き（クリックで選択）</span>
    <span class="badge badge-red">■ 使用中</span>
    <span class="badge" style="background:#E3F2FD; color:#1565C0;">■ 選択中</span>
  </div>
  <div class="seat-grid">
    <% for (Seat s : areaSeats) {
      boolean occupied = seatUsage != null && seatUsage.containsKey(s.getSeatId());
      String uname = (occupied && seatUserNameMap != null && seatUserNameMap.containsKey(s.getSeatId()))
        ? seatUserNameMap.get(s.getSeatId()) : "";
      String udept = (occupied && seatUserDeptMap != null && seatUserDeptMap.containsKey(s.getSeatId()))
        ? seatUserDeptMap.get(s.getSeatId()) : "";
      String hoverInfo = (udept != null && !udept.isEmpty()) ? (udept + " " + uname) : uname;
    %>
    <% if (!occupied) { %>
      <label style="margin:0; cursor:pointer;">
        <input type="checkbox" name="seatIds" value="<%= s.getSeatId() %>"
          style="display:none;" onchange="toggleSeat(this)">
        <div class="seat-btn available" id="seatBtn<%= s.getSeatId() %>">
          <span class="seat-id"><%= s.getSeatName() %></span>
          <span class="seat-user">空き</span>
        </div>
      </label>
    <% } else { %>
      <!-- カーソルを合わせると title 属性で「部署名 氏名」がツールチップ表示される -->
      <div class="seat-btn occupied" style="cursor:not-allowed;" title="<%= hoverInfo %>">
        <span class="seat-id"><%= s.getSeatName() %></span>
        <span class="seat-user"><%= uname %><% if (udept != null && !udept.isEmpty()) { %><span style="font-size:11px; opacity:.8;">（<%= udept %>）</span><% } %></span>
      </div>
    <% } %>
  <% } %>
</div>

<div class="form-group" style="margin-top:16px; max-width:480px;">
  <label for="adminMemo">メモ（任意・この予約全件に共通で設定されます）</label>
  <textarea id="adminMemo" name="memo" rows="2" placeholder="例：来客対応のため など" style="width:100%;"></textarea>
</div>

<div style="margin-top:20px;">
  <button type="submit" class="btn btn-primary btn-lg">
    選択した席で予約する
  </button>
</div>
</div>
<% } else if (selArea != null && !selArea.isEmpty()) { %>
<div class="step-section">
  <div class="alert alert-info">エリアを選択すると座席が表示されます。</div>
</div>
<% } %>

</form>

<div style="margin-top:20px;">
  <a href="<%= ctx %>/ControlServlet?action=adminMenu" class="btn btn-secondary">&larr; 管理メニューへ戻る</a>
</div>
</div>
</div>

<script>

```

```

// 社員一覧をJSの配列として用意しておき、検索・追加をブラウザ側だけで行えるようにする
var ACCOUNTS = [
  <% if (accountRows != null) {
    boolean first = true;
    for (Map<String, Object> row : accountRows) {
      if (!first) { %>, <% }
      first = false;
    %> { userId: "<%= row.get("userId") %>", name: "<%= row.get("name") %>", bName: "<%= row.get("bName") %>" }
  <% }
    } %>
  ];

var selectedIndividual = null; // 個人モード：1名のみ
var selectedMeeting = {}; // 会議モード：userId をキーにした複数名

function findAccount(userId) {
  for (var i = 0; i < ACCOUNTS.length; i++) {
    if (ACCOUNTS[i].userId === userId) return ACCOUNTS[i];
  }
  return null;
}

function searchAccounts(mode) {
  var cap = (mode === 'individual') ? 'Individual' : 'Meeting';
  var dept = document.getElementById('searchDept' + cap).value;
  var name = document.getElementById('searchName' + cap).value.trim();
  var box = document.getElementById('searchResult' + cap);
  box.innerHTML = '';

  var hits = ACCOUNTS.filter(function(acc) {
    if (dept !== '' && acc.bName !== dept) return false;
    if (name !== '' && acc.name.indexOf(name) === -1) return false;
    return true;
  });

  if (hits.length === 0) {
    box.innerHTML = '<p style="color:#9E9E9E; font-size:13px;">該当する社員が見つかりませんでした。条件を変えて検索してください。</p>';
    return;
  }

  hits.forEach(function(acc) {
    var row = document.createElement('div');
    row.className = 'account-row';
    row.style.display = 'flex';
    row.style.alignItems = 'center';
    row.style.gap = '12px';

    var label = document.createElement('span');
    label.textContent = acc.bName + 'の' + acc.name + ' (' + acc.userId + ')';
    row.appendChild(label);

    var addBtn = document.createElement('button');
    addBtn.type = 'button';
    addBtn.className = 'btn btn-sm btn-primary';
    addBtn.textContent = '追加';
    addBtn.onclick = (function(uid) { return function() { addPerson(mode, uid); }; })(acc.userId);
    row.appendChild(addBtn);

    box.appendChild(row);
  });
}

```

```

function addPerson(mode, userId) {
  var acc = findAccount(userId);
  if (!acc) return;

  if (mode === 'individual') {
    selectedIndividual = acc; // 個人は1名だけなので、後から選び直すと置き換わる
  } else if (!selectedMeeting[userId]) {
    selectedMeeting[userId] = acc; // 同じ人を二重に追加しない
  }
  renderSelected(mode);
}

function removePerson(mode, userId) {
  if (mode === 'individual') {
    selectedIndividual = null;
  } else {
    delete selectedMeeting[userId];
  }
  renderSelected(mode);
}

function renderSelected(mode) {
  var cap = (mode === 'individual') ? 'Individual' : 'Meeting';
  var box = document.getElementById('selected' + cap);
  var list = (mode === 'individual')
    ? (selectedIndividual ? [selectedIndividual] : [])
    : Object.keys(selectedMeeting).map(function(uid) { return selectedMeeting[uid]; });

  box.innerHTML = '';
  if (list.length === 0) {
    box.innerHTML = '<p style="color:#9E9E9E; font-size:13px;">まだ誰も追加されていません。検索して「追加」を押してください。</p>';
  } else {
    list.forEach(function(acc) {
      var chip = document.createElement('span');
      chip.className = 'badge';
      chip.style.background = '#E0F2F1';
      chip.style.color = '#00695C';
      chip.textContent = acc.bName + 'の' + acc.name;

      var removeBtn = document.createElement('a');
      removeBtn.href = 'javascript:void(0)';
      removeBtn.style.marginLeft = '8px';
      removeBtn.style.color = '#C62828';
      removeBtn.style.fontWeight = '700';
      removeBtn.textContent = 'x';
      removeBtn.onclick = (function(uid) { return function() { removePerson(mode, uid); }; })(acc.userId);

      chip.appendChild(removeBtn);
      box.appendChild(chip);
    });
  }

  // フォーム送信用の隠しinputを作り直す
  var hidden = document.getElementById('hidden' + cap);
  hidden.innerHTML = '';
  if (mode === 'individual') {
    if (selectedIndividual) {
      hidden.innerHTML = '<input type="hidden" name="targetUserId" value="' + selectedIndividual.userId + '">';
    }
  } else {
    list.forEach(function(acc) {
      hidden.innerHTML += '<input type="hidden" name="targetUserIds" value="' + acc.userId + '">';
    });
  }
}

```

```

    });
  }
}

renderSelected('individual');
renderSelected('meeting');

function setMode(mode) {
  document.getElementById('modeInput').value = mode;
  document.getElementById('modeCardIndividual').classList.toggle('active', mode === 'individual');
  document.getElementById('modeCardMeeting').classList.toggle('active', mode === 'meeting');
  document.getElementById('sectionIndividual').style.display = mode === 'individual' ? '' : 'none';
  document.getElementById('sectionMeeting').style.display = mode === 'meeting' ? '' : 'none';
  // モード切り替え時は座席選択をリセット（個人・会議で選択数の意味が変わるため）
  document.querySelectorAll('input[name="seatIds"]:checked').forEach(function(cb) {
    cb.checked = false;
    var btn = document.getElementById('seatBtn' + cb.value);
    if (btn) { btn.classList.remove('selected'); btn.classList.add('available'); }
  });
}

setMode('individual'); // 初期表示

function toggleSeat(cb) {
  // 個人の代理予約は1名・1席のため、他の座席が選択済みなら解除する
  if (document.getElementById('modeInput').value === 'individual' && cb.checked) {
    document.querySelectorAll('input[name="seatIds"]:checked').forEach(function(other) {
      if (other !== cb) {
        other.checked = false;
        var otherBtn = document.getElementById('seatBtn' + other.value);
        if (otherBtn) { otherBtn.classList.remove('selected'); otherBtn.classList.add('available'); }
      }
    });
  }
  var btn = document.getElementById('seatBtn' + cb.value);
  if (cb.checked) {
    btn.classList.remove('available'); btn.classList.add('selected');
  } else {
    btn.classList.remove('selected'); btn.classList.add('available');
  }
}

// 拠点・フロア・エリアのプルダウンを変えるたびにページ上部へ戻されてしまうのを防ぐため、
// 遷移前のスクロール位置を覚えておき、再読み込み後（画像読み込み完了後）に同じ位置へ戻す
var ADMIN_RESERVE_SCROLL_KEY = 'adminReserveScrollY';
(function() {
  var saved = sessionStorage.getItem(ADMIN_RESERVE_SCROLL_KEY);
  if (saved === null) return;
  sessionStorage.removeItem(ADMIN_RESERVE_SCROLL_KEY);
  window.addEventListener('load', function() {
    window.scrollTo(0, parseInt(saved, 10));
  });
})();

function reloadWith(paramName, value) {
  sessionStorage.setItem(ADMIN_RESERVE_SCROLL_KEY, String(window.scrollY));
  var url = '<%= request.getContextPath() %>/ControlServlet?action=adminReserve';
  var base = document.querySelector('[name=base]') ? document.querySelector('[name=base]').value : '';
  var floor = document.querySelector('[name=floor]') ? document.querySelector('[name=floor]').value : '';
  var area = document.querySelector('[name=area]') ? document.querySelector('[name=area]').value : '';
  var date = document.querySelector('[name=date]') ? document.querySelector('[name=date]').value : '';
  if (paramName === 'base') { base = value; floor = ''; area = ''; }
  if (paramName === 'floor') { floor = value; area = ''; }
  if (paramName === 'area') { area = value; }
}

```



```
location.href = url + '&base=' + encodeURIComponent(base)
               + '&floor=' + encodeURIComponent(floor)
               + '&area='  + encodeURIComponent(area)
               + '&date='  + encodeURIComponent(date);
}
</script>
</body>
</html>
```

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.List, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>マスタ更新 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/headerAdmin.jspf" %>

<div class="container">
    <div class="card">
        <h2>座席マスタ更新</h2>

        <!-- 完了メッセージ -->
        <% if (request.getAttribute("message") != null) { %>
        <div class="alert alert-success">&#10003; <%= request.getAttribute("message") %></div>
        <% } %>

        <%
        List<Seat> seats    = (List<Seat>) request.getAttribute("seats");
        List<String> bases  = (List<String>) request.getAttribute("bases");
        Seat editSeat      = (Seat) request.getAttribute("editSeat");
        String selectedBase = (String) request.getAttribute("selectedBase");
        String officeImage  = (String) request.getAttribute("officeImage");
        String ctx = request.getContextPath();
        %>

        <p style="font-size:12px; color:#757575; margin-bottom:10px;">
            ① オフィスを選ぶと、フロアマップ画像とそのオフィスの座席だけを表示できます（見やすさ・確認用の絞り込みです）。
        </p>

        <!-- ① オフィスを選んで表示・絞り込み -->
        <form action="<%= ctx %>/ControlServlet" method="get" style="margin-bottom:14px;">
            <input type="hidden" name="action" value="masterUpdate">
            <div class="filter-row">
                <div class="form-group">
                    <label>① オフィスを選択</label>
                    <select name="filterBase" onchange="submitMasterFilterForm(this)">
                        <option value="">-- すべてのオフィスを表示 --</option>
                        <% if (bases != null) for (String b : bases) { %>
                        <option value="<%= b %>" <%= b.equals(selectedBase) ? "selected" : "" %>><%= b %></option>
                        <% } %>
                    </select>
                </div>
            </div>
        </form>

        <!-- 選んだオフィスのフロアマップ画像（画像を見ながら座席を確認・追加できるように） -->
        <% if (officeImage != null) { %>
        <div style="margin-bottom:16px;">
            のフロアマップ"
                style="max-width:100%; border:1px solid #ECEFF1; border-radius:8px;">
        </div>
        <% } else if (selectedBase != null && !selectedBase.isEmpty()) { %>
        <p style="color:#9E9E9E; font-size:12px; margin-bottom:16px;">
            ※「<%= selectedBase %>」のフロアマップ画像はまだ登録されていません。画像を用意したら
            <code>SeatsDAO</code> の <code>OFFICE_IMAGES</code> に追加すると、ここにも表示されます。
        </p>

```

```

<% } %>

<!-- 座席一覧 (①で選んだオフィスのみ表示。未選択ならすべて表示) -->
<h3 style="margin-bottom:8px; color:var(--teal-dark);">
  <%= (selectedBase != null && !selectedBase.isEmpty()) ? (selectedBase + "の座席一覧") : "座席一覧 (全オフィス)"
%>
  <span style="font-size:12px; color:#9E9E9E; font-weight:400;"> (<%= seats != null ? seats.size() : 0 %> 席) </s
pan>
</h3>
<!--
件数が増えてもページ全体が縦に伸び続けないよう、一覧をカード内の
スクロール領域（最大高さ固定 + ヘッダー追従）に収める。
「①オフィスを選択」で絞り込めば、その拠点だけの短い一覧として確認できる。
-->
<div style="max-height:480px; overflow-y:auto; border:1px solid #ECEFF1; border-radius:8px; margin-bottom:24p
x;">
<table class="table" style="margin-bottom:0;">
  <thead>
    <tr>
      <th style="position:sticky; top:0; background:#fff; z-index:1;">ID</th>
      <th style="position:sticky; top:0; background:#fff; z-index:1;">拠点</th>
      <th style="position:sticky; top:0; background:#fff; z-index:1;">フロア</th>
      <th style="position:sticky; top:0; background:#fff; z-index:1;">エリア</th>
      <th style="position:sticky; top:0; background:#fff; z-index:1;">席名</th>
      <th style="position:sticky; top:0; background:#fff; z-index:1; width:140px;">操作</th>
    </tr>
  </thead>
  <tbody>
    <%
      String rowBase = null;
      if (seats != null) for (Seat s : seats) {
        boolean newBase = !s.getBaseName().equals(rowBase);
        rowBase = s.getBaseName();
        if (newBase && (selectedBase == null || selectedBase.isEmpty())) {
    %>
    <!-- 全オフィス表示時は拠点の切れ目が見出し行を挟む -->
    <tr style="background:#F5F5F5;">
      <td colspan="6" style="font-weight:700; color:var(--teal-dark);"><%= rowBase %></td>
    </tr>
    <% } %>
    <tr>
      <td><%= s.getSeatId() %></td>
      <td><%= s.getBaseName() %></td>
      <td><%= s.getFName() %></td>
      <td><%= s.getAreaName() %></td>
      <td><%= s.getSeatName() %></td>
      <td>
        <!-- 更新リンク → 編集フォームにIDを渡す -->
        <a href="<%= ctx %>/ControlServlet?action=masterEdit&seatId=<%= s.getSeatId() %>&filterBase=<%= selected
Base != null ? selectedBase : "" %>"
          class="btn btn-primary btn-sm">更新</a>
        <!-- 削除ボタン (POSTフォームを外部に配置し form= で紐づけ) -->
        <button type="submit" form="delForm<%= s.getSeatId() %>" class="btn btn-danger btn-sm"
          onclick="return confirm('座席ID <%= s.getSeatId() %> を削除しますか?')">削除</button>
      </td>
    </tr>
  </tbody>
</table>
</div>

<!-- 削除フォーム (テーブル外に配置) -->

```

```

<% if (seats != null) for (Seat s : seats) { %>
<form id="delForm"<%= s.getSeatId() %>" action="<%= ctx %>/ControlServlet" method="post" style="display:none;">
  <input type="hidden" name="action" value="doMasterDelete">
  <input type="hidden" name="seatId" value="<%= s.getSeatId() %>">
</form>
<% } %>

<!-- 更新フォーム (masterEditで対象座席が設定された場合に表示) -->
<% if (editSeat != null) { %>
<div class="card" style="background:#E3F2FD; border:2px solid #42A5F5; margin-bottom:20px;">
  <h2 style="color:#1565C0;">座席 ID <%= editSeat.getSeatId() %> を更新</h2>
  <form action="<%= ctx %>/ControlServlet" method="post">
    <input type="hidden" name="action" value="doMasterUpdate">
    <input type="hidden" name="seatId" value="<%= editSeat.getSeatId() %>">
    <div class="filter-row">
      <div class="form-group">
        <label>拠点</label>
        <input type="text" name="baseName" value="<%= editSeat.getBaseName() %>" required>
      </div>
      <div class="form-group">
        <label>フロア</label>
        <input type="text" name="fName" value="<%= editSeat.getFName() %>" required>
      </div>
      <div class="form-group">
        <label>エリア</label>
        <input type="text" name="areaName" value="<%= editSeat.getAreaName() %>" required>
      </div>
      <div class="form-group">
        <label>席名</label>
        <input type="text" name="seatName" value="<%= editSeat.getSeatName() %>" required>
      </div>
      <div>
        <button type="submit" class="btn btn-primary">更新する</button>
        <a href="<%= ctx %>/ControlServlet?action=masterUpdate&filterBase=<%= selectedBase != null ? selectedBase : "" %>" class="btn btn-secondary">キャンセル</a>
      </div>
    </div>
  </form>
</div>
<% } %>

<!-- 新規追加フォーム -->
<div class="card" style="background:#F9FBE7; border:2px solid #AFB42B;">
  <h2 style="color:#558B2F;">② 座席を追加する</h2>
  <p style="font-size:12px; color:#757575; margin-bottom:10px;">
    既存のオフィスに座席を増やすときは「拠点」で一覧から選び、まだ無いオフィスを増やしたいときは
    「+ 新しいオフィスを追加する」を選んでオフィス名を入力してください。
    ※オフィスという独立したマスタ行はなく、座席を1件追加した時点でそのオフィス名が新しく登場する仕組みです (③参照)。
  </p>
  <form action="<%= ctx %>/ControlServlet" method="post" id="addSeatForm">
    <input type="hidden" name="action" value="doMasterAdd">
    <div class="filter-row">
      <div class="form-group">
        <label>拠点</label>
        <select name="baseName" id="baseSelect" onchange="onBaseSelectChange()">
          <% if (bases != null) for (String b : bases) { %>
            <option value="<%= b %>" <%= b.equals(selectedBase) ? "selected" : "" %>><%= b %></option>
          <% } %>
          <option value="__NEW__">+ 新しいオフィスを追加する</option>
        </select>
      </div>
      <div class="form-group" id="newBaseGroup" style="display:none;">
        <label>新しいオフィス名</label>

```

```

        <input type="text" name="newBaseName" id="newBaseName" placeholder="例: 渋谷オフィス">
    </div>
    <div class="form-group">
        <label>フロア</label>
        <input type="text" name="fName" placeholder="例: 3F" required>
    </div>
    <div class="form-group">
        <label>エリア</label>
        <input type="text" name="areaName" placeholder="例: Cエリア" required>
    </div>
    <div class="form-group">
        <label>席名</label>
        <input type="text" name="seatName" placeholder="例: C-01" required>
    </div>
    <div>
        <button type="submit" class="btn btn-green">追加する</button>
    </div>
</div>
</form>
<p style="font-size:11px; color:#9E9E9E; margin-top:10px;">
    ③ 新しいオフィスを増やしたときは、フロアマップ画像も用意して
    <code>SeatsDAO</code> の <code>OFFICE_IMAGES</code> に登録すると、この画面・座席利用状況・代理予約の画面にも
    画像が表示されるようになります（手順は <code>SeatsDAO.java</code> 冒頭のコメントを参照）。
</p>
</div>

<script>
    // 「拠点」で「+ 新しいオフィスを追加する」を選んだときだけ、
    // 新しいオフィス名の入力欄を表示／必須にする（vanilla JS、フレームワーク不使用）
    function onBaseSelectChange() {
        var sel = document.getElementById('baseSelect');
        var grp = document.getElementById('newBaseGroup');
        var isNew = (sel.value === '__NEW__');
        grp.style.display = isNew ? '' : 'none';
        document.getElementById('newBaseName').required = isNew;
    }
    onBaseSelectChange();
</script>

<div style="margin-top:10px;">
    <a href="%= ctx %>/ControlServlet?action=adminMenu" class="btn btn-secondary">&larr; 管理メニューへ戻る</a>
</div>
</div>
</div>

<script>
    // 更新・追加・削除・絞り込みのたびにページ上部へ戻されてしまうのを防ぐため、
    // 送信前のスクロール位置を覚えておき、再読み込み後に同じ位置へ戻す。
    //
    // つまずきポイント1: <select onchange="this.form.submit()"> のように
    //   フォームの submit() メソッドを直接呼び出すと "submit" イベントは発火しない
    //   (addEventListener('submit', ...) では拾えない)。そのため絞り込み用セレクトは
    //   保存してから送信する関数を直接 onchange から呼び形にしている。
    // つまずきポイント2: ページ読み込み直後（スクリプト実行時点）に scrollTo すると、
    //   画像（フロアマップ）の読み込みで後からレイアウトが変わり、
    //   結局トップへ戻ってしまう。window の load イベント（画像読み込み完了後）まで待つ。
    var MASTER_SCROLL_KEY = 'masterUpdateScrollY';

    function submitMasterFilterForm(sel) {
        sessionStorage.setItem(MASTER_SCROLL_KEY, String(window.scrollY));
        sel.form.submit();
    }

```

```
document.querySelectorAll('form').forEach(function(f) {
  f.addEventListener('submit', function() {
    sessionStorage.setItem(MASTER_SCROLL_KEY, String(window.scrollY));
  });
});

(function() {
  var saved = sessionStorage.getItem(MASTER_SCROLL_KEY);
  if (saved === null) return;
  sessionStorage.removeItem(MASTER_SCROLL_KEY);
  window.addEventListener('load', function() {
    window.scrollTo(0, parseInt(saved, 10));
  });
})();
</script>
</body>
</html>
```

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.*, entity.Seat, dao.SeatsDAO" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>座席利用状況 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/headerAdmin.jspf" %>

<div class="container">
    <div class="card">
        <h2>座席利用状況確認</h2>

        <%
            List<String> bases = (List<String>) request.getAttribute("bases");
            List<String> floors = (List<String>) request.getAttribute("floors");
            List<String> areas = (List<String>) request.getAttribute("areas");
            String filterBase = (String) request.getAttribute("filterBase");
            String filterFloor = (String) request.getAttribute("filterFloor");
            String filterArea = (String) request.getAttribute("filterArea");
            String filterDate = (String) request.getAttribute("filterDate");
            LinkedHashMap<String, String> dateOptions = (LinkedHashMap<String, String>) request.getAttribute("dateOptions");

            String ctx = request.getContextPath();

            String myDeptName = (String) request.getAttribute("myDeptName");
            @SuppressWarnings("unchecked")
            List<Map<String, Object>> deptTodayList = (List<Map<String, Object>>) request.getAttribute("deptTodayList");
        %>

        <!-- 同じ部署の本日の出社者一覧（自分の部署のメンバーが今日どこにいるかをひと目で確認） -->
        <% if (myDeptName != null) { %>
        <div style="background:#F5F5F5; border-radius:8px; padding:12px 16px; margin-bottom:16px;">
            <strong style="font-size:13px; color:#424242;"><%= myDeptName %>の本日の出社者</strong>
            <% if (deptTodayList == null || deptTodayList.isEmpty()) { %>
            <p style="color:#9E9E9E; font-size:13px; margin:6px 0 0 0;">本日「出社」の予定があるメンバーはいません。</p>
            <% } else { %>
            <div class="summary-row" style="margin-top:8px; gap:8px;">
                <% for (Map<String, Object> row : deptTodayList) {
                    Seat dSeat = (Seat) row.get("seat");
                    String dSeatLabel = dSeat != null
                        ? dSeat.getBaseName() + " " + dSeat.getFName() + " " + dSeat.getAreaName() + " " + dSeat.getSeatName()
                        : "座席未定";

                    %>
                    <span class="badge badge-gray"><%= row.get("name") %> <span style="color:#00695C;"><%= dSeatLabel %></span></span>
                <% } %>
            </div>
            <% } %>
        </div>
        <% } %>
        </div>
        <% } %>

        <p style="font-size:12px; color:#757575; margin-bottom:10px;">
            「〇〇オフィスの〇〇エリア」のように、拠点 → フロア → エリアの順に具体的な場所を選んで確認します。
        </p>

        <!-- 絞り込みフォーム（拠点 → フロア → エリアの順に選択。前の項目を選ぶまで次は選べない） -->

```

```

<form action="<%= ctx %>/ControlServlet" method="get">
  <input type="hidden" name="action" value="doSeatStatus">
  <div class="filter-row">
    <div class="form-group">
      <label>① 拠点</label>
      <select name="filterBase" onchange="this.form.submit()">
        <option value="">-- 拠点を選択 --</option>
        <% if (bases != null) for (String b : bases) { %>
          <option value="<%= b %>" <%= b.equals(filterBase) ? "selected" : "" %>><%= b %></option>
        <% } %>
      </select>
    </div>
    <div class="form-group">
      <label>② フロア</label>
      <select name="filterFloor" onchange="this.form.submit()" <%= (floors == null) ? "disabled" : "" %>>
        <option value="">-- フロアを選択 --</option>
        <% if (floors != null) for (String f : floors) { %>
          <option value="<%= f %>" <%= f.equals(filterFloor) ? "selected" : "" %>><%= f %></option>
        <% } %>
      </select>
    </div>
    <div class="form-group">
      <label>③ エリア</label>
      <select name="filterArea" onchange="this.form.submit()" <%= (areas == null) ? "disabled" : "" %>>
        <option value="">-- エリアを選択 --</option>
        <% if (areas != null) for (String a : areas) { %>
          <option value="<%= a %>" <%= a.equals(filterArea) ? "selected" : "" %>><%= a %></option>
        <% } %>
      </select>
    </div>
    <div class="form-group">
      <label>④ 日付（2週間前～2週間後まで確認できます）</label>
      <select name="filterDate" onchange="this.form.submit()">
        <% if (dateOptions != null) for (Map.Entry<String, String> e : dateOptions.entrySet()) { %>
          <option value="<%= e.getKey() %>" <%= e.getKey().equals(filterDate) ? "selected" : "" %>><%= e.getValue
() %></option>
        <% } %>
      </select>
    </div>
  </div>
</form>

<!-- 拠点ごとのフロアマップ画像（拠点を選べると該当オフィスの画像に切り替わる） -->
<%
  String officeImage = (filterBase != null && !filterBase.isEmpty())
    ? new SeatsDAO().getOfficeImage(filterBase) : null;
%>
<% if (officeImage != null) { %>
  <div style="margin:16px 0;">
    のフロアマップ"
      style="max-width:100%; border:1px solid #ECEFF1; border-radius:8px;">
  </div>
<% } %>

<%
  List<Seat> filteredSeats = (List<Seat>) request.getAttribute("filteredSeats");
  Map<Integer, String> seatUsageMap = (Map<Integer, String>) request.getAttribute("seatUsageMap");
  Map<Integer, String> seatUserNameMap = (Map<Integer, String>) request.getAttribute("seatUserNameMap");
  Map<Integer, String> seatUserDeptMap = (Map<Integer, String>) request.getAttribute("seatUserDeptMap");
  Object usedObj = request.getAttribute("usedCount");
  Object freeObj = request.getAttribute("freeCount");
  long usedCount = usedObj instanceof Long ? (Long)usedObj : (usedObj != null ? ((Integer)usedObj).longValue() :
0);

```



```

    long freeCount = freeObj instanceof Long ? (Long)freeObj : (freeObj != null ? ((Integer)freeObj).longValue() :
0);
    %>

<% if (filterArea != null && !filterArea.isEmpty()) { %>
<!-- 結果見出し：常に「○○オフィスの○○ ○○エリア」のように具体的な場所を一意に表示する -->
<h3 style="margin:18px 0 10px; color:#00695C;">
    <%= filterBase %>の<%= filterFloor %> <%= filterArea %>
</h3>

<!-- サマリ -->
<div class="summary-row" style="margin-bottom:16px;">
    <span class="badge badge-gray">対象 <%= filteredSeats.size() %> 席</span>
    <span class="badge badge-red">使用中 <%= usedCount %> 席</span>
    <span class="badge badge-green">空き <%= freeCount %> 席</span>
</div>

<!-- 凡例 -->
<div class="summary-row" style="font-size:12px;">
    <span class="badge badge-green">■ 空き</span>
    <span class="badge badge-red">■ 使用中</span>
</div>

<!--
座席マップ：席のボタンは76×54pxと小さく「氏名（部署名）」を入れると見切れてしまうため、
ここでは座席名と状態だけを示すミニマップに留め、カーソルを合わせると title 属性で
「部署名 氏名」をツールチップ表示する（誰がどこにいるかの全体像をひと目で把握する用途）。
詳しい内訳は、すぐ下の「使用中の座席一覧」で省略なしに確認できる。
-->
<div class="seat-grid">
    <% for (Seat s : filteredSeats) {
        boolean occupied = seatUsageMap != null && seatUsageMap.containsKey(s.getSeatId());
        String uname = (seatUserNameMap != null && seatUserNameMap.containsKey(s.getSeatId()))
            ? seatUserNameMap.get(s.getSeatId()) : "";
        String udept = (seatUserDeptMap != null && seatUserDeptMap.containsKey(s.getSeatId()))
            ? seatUserDeptMap.get(s.getSeatId()) : "";
        String hoverInfo = occupied ? ((udept != null && !udept.isEmpty()) ? (udept + " " + uname) : uname) : "空
き";
        %>
        <div class="seat-btn <%= occupied ? "occupied" : "available" %>" style="cursor:default;" title="<%= hoverInf
o %>">
            <span class="seat-id"><%= s.getSeatName() %></span>
            <span class="seat-user"><%= occupied ? "使用中" : "空き" %></span>
        </div>
        <% } %>
    </div>

<!-- 使用中の座席一覧：誰がどの座席を使っているかを、省略せず一覧で確認できるようにする -->
<% if (usedCount > 0) { %>
<h4 style="margin:22px 0 8px; color:#424242; font-size:14px;">使用中の座席一覧 (<%= usedCount %> 件) </h4>
<table class="table" style="margin-bottom:8px;">
    <thead>
        <tr><th style="width:140px;">座席</th><th>部署</th><th>氏名</th></tr>
    </thead>
    <tbody>
        <% for (Seat s : filteredSeats) {
            if (seatUsageMap == null || !seatUsageMap.containsKey(s.getSeatId())) continue;
            String uname = (seatUserNameMap != null && seatUserNameMap.containsKey(s.getSeatId()))
                ? seatUserNameMap.get(s.getSeatId()) : "";
            String udept = (seatUserDeptMap != null && seatUserDeptMap.containsKey(s.getSeatId()))
                ? seatUserDeptMap.get(s.getSeatId()) : "";
            %>
            <tr>

```

```

        <td><span class="badge badge-red" style="white-space:nowrap;"><%= s.getSeatName() %></span></td>
        <td><%= udept %></td>
        <td><%= uname %></td>
    </tr>
    <% } %>
</tbody>
</table>
<% } %>
<% } else { %>
<!-- 拠点・フロア・エリアがすべて選ばれるまでは、案内メッセージのみ表示する -->
<p style="color:#9E9E9E; padding:24px 0; text-align:center;">
    拠点・フロア・エリアを順番に選択すると、その場所の座席利用状況が表示されます。
</p>
<% } %>

<div style="margin-top:24px;">
    <a href="<%= ctx %>/ControlServlet?action=adminMenu" class="btn btn-secondary">&larr; 管理メニューへ戻る</a>
</div>
</div>
</body>
</html>

```

9. JSP（検索画面）・設定・CSS

webapp/WEB-INF/jsp/nameSearch.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.*, entity.Account, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>氏名検索 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<%
    String menuAction = Boolean.TRUE.equals(session.getAttribute("isAdmin")) ? "adminMenu" : "menu";
%>

<div class="container">
    <div class="card">
        <h2>氏名検索</h2>

        <form action="<%= request.getContextPath() %>/ControlServlet" method="get">
            <input type="hidden" name="action" value="doNameSearch">
            <div style="display:flex; gap:10px; align-items:flex-end;">
                <div class="form-group" style="flex:1; margin:0;">
                    <label>氏名（部分一致）</label>
                    <input type="text" name="keyword"
                        value="<%= request.getAttribute("keyword") != null ? request.getAttribute("keyword") : "" %>"
                        placeholder="例: 田中" autofocus>
                </div>
                <button type="submit" class="btn btn-primary">
                    検索
                </button>
            </div>
        </form>

        <% List<Map<String, Object>> results = (List<Map<String, Object>>) request.getAttribute("results"); %>
        <% if (results != null) { %>
        <div style="margin-top:20px;">
            <p style="font-size:13px; color:#757575; margin-bottom:8px;"><%= results.size() %> 件見つかりました</p>
            <table class="table">
                <thead>
                    <tr><th>ユーザーID</th><th>氏名</th><th>部門</th><th>本日の行先</th><th>本日の予約座席</th></tr>
                </thead>
                <tbody>
                    <% for (Map<String, Object> row : results) {
                        Account a = (Account) row.get("account");
                        Seat seat = (Seat) row.get("seat");
                    %>
                    <tr>
                        <td><%= a.getUserId() %></td>
                        <td><%= a.getName() %></td>
                        <td><%= row.get("bName") %></td>
                        <td><span class="badge badge-green"><%= row.get("todayStatus") %></span></td>
                        <td>
                            <% if (seat != null) { %>
                                <%= seat.getBaseName() %> <%= seat.getFName() %> <%= seat.getAreaName() %> <%= seat.getSeatName() %>
                            <% } else { %>
                                <%= "座席なし" %>
                            <% } %>
                        </td>
                    </tr>
                </tbody>
            </table>
        </div>
        <% } %>
    </div>
</div>
```

```
        <span style="color:var(--text-muted);">-</span>
      <% } %>
    </td>
  </tr>
<% } %>
</tbody>
</table>
</div>
<% } %>

<div style="margin-top:20px;">
  <a href="<%= request.getContextPath() %>/ControlServlet?action=<%= menuAction %>" class="btn btn-secondary">&l
arr; メニューへ戻る</a>
</div>
</div>
</body>
</html>
```

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.*, entity.Account, entity.Location, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>部門検索 - 座・Navic</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<%
String menuAction = Boolean.TRUE.equals(session.getAttribute("isAdmin")) ? "adminMenu" : "menu";
%>

<div class="container">
<div class="card">
<h2>部門検索</h2>

<%
List<Location> departments = (List<Location>) request.getAttribute("departments");
Integer selectedBId = (Integer) request.getAttribute("selectedBId");
String selectedBName = (String) request.getAttribute("selectedBName");
%>

<form action="<%= request.getContextPath() %>/ControlServlet" method="get">
<input type="hidden" name="action" value="doDepartmentSearch">
<div style="display:flex; gap:10px; align-items:flex-end;">
<div class="form-group" style="flex:1; margin:0;">
<label>部門を選択</label>
<select name="bId">
<option value="">-- 部門を選んでください --</option>
<% if (departments != null) for (Location dept : departments) { %>
<option value="<%= dept.getBId() %>" <%= (selectedBId != null && selectedBId == dept.getBId()) ? "selected" : "" %>
<%= dept.getBName() %>
</option>
<% } %>
</select>
</div>
<button type="submit" class="btn btn-primary">
検索
</button>
</div>
</form>

<% List<Map<String, Object>> results = (List<Map<String, Object>>) request.getAttribute("results"); %>
<% if (results != null) { %>
<div style="margin-top:20px;">
<p style="font-size:13px; color:#757575; margin-bottom:8px;">
「<%= selectedBName %>」のメンバー：<%= results.size() %> 名
</p>
<table class="table">
<thead>
<tr><th>ユーザーID</th><th>氏名</th><th>本日の行先</th><th>本日の予約座席</th></tr>
</thead>
<tbody>
<% for (Map<String, Object> row : results) {
Account a = (Account) row.get("account");
Seat seat = (Seat) row.get("seat");

```

```

%>
<tr>
  <td><%= a.getUserId() %></td>
  <td><%= a.getName() %></td>
  <td><span class="badge badge-green"><%= row.get("todayStatus") %></span></td>
  <td>
    <% if (seat != null) { %>
      <%= seat.getBaseName() %> <%= seat.getFName() %> <%= seat.getAreaName() %> <%= seat.getSeatName() %
>
      <% } else { %>
        <span style="color:var(--text-muted);">-</span>
      <% } %>
    </td>
  </tr>
<% } %>
</tbody>
</table>
</div>
<% } %>

<div style="margin-top:20px;">
  <a href="<%= request.getContextPath() %>/ControlServlet?action=<%= menuAction %>" class="btn btn-secondary">&l
arr; ✕メニューへ戻る</a>
</div>
</div>
</div>
</body>
</html>

```

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
  version="4.0">

  <display-name>座・Navi（Za-Navi） 座席管理システム</display-name>

  <!-- 文字コードフィルター（すべてのリクエストを UTF-8 で処理） -->
  <filter>
    <filter-name>CharacterEncodingFilter</filter-name>
    <filter-class>org.apache.catalina.filters.SetCharacterEncodingFilter</filter-class>
    <init-param>
      <param-name>encoding</param-name>
      <param-value>UTF-8</param-value>
    </init-param>
    <init-param>
      <param-name>ignore</param-name>
      <param-value>>false</param-value>
    </init-param>
  </filter>
  <filter-mapping>
    <filter-name>CharacterEncodingFilter</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>

  <!-- ControlServlet を /ControlServlet にマッピング -->
  <servlet>
    <servlet-name>ControlServlet</servlet-name>
    <servlet-class>control.ControlServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>ControlServlet</servlet-name>
    <url-pattern>/ControlServlet</url-pattern>
  </servlet-mapping>

  <!-- デフォルトページ：ログイン画面 -->
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>

  <!-- セッションタイムアウト 30分 -->
  <session-config>
    <session-timeout>30</session-timeout>
  </session-config>

</web-app>
```

```

/* =====
Za-Navi 座席管理システム - スタイルシート
基調色: ティールブルー (#00897B)
===== */
:root {
  --teal:      #00897B;
  --teal-light: #4DB6AC;
  --teal-dark: #00695C;
  --bg:        #ECEFF1;
  --card:      #FFFFFF;
  --text:      #212121;
  --text-muted: #757575;
  --green:     #4CAF50;
  --green-dk:  #388E3C;
  --red:       #F44336;
  --red-dk:    #C62828;
  --yellow:    #FFC107;
  --orange:    #FF9800;
  --shadow:    0 2px 8px rgba(0,0,0,0.12);
}

* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: 'Segoe UI', Meiryo, 'Hiragino Sans', sans-serif;
  background: var(--bg);
  color: var(--text);
  font-size: 14px;
  line-height: 1.5;
}

/* — ヘッダー — */
.header {
  background: var(--teal);
  color: #fff;
  height: 56px;
  padding: 0 24px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  box-shadow: 0 2px 6px rgba(0,0,0,0.2);
  position: sticky;
  top: 0;
  z-index: 100;
}

.header .logo      { font-size: 20px; font-weight: 700; letter-spacing: 2px; }
.header .logo span { font-size: 12px; font-weight: 400; margin-left: 8px; opacity: .8; }
.header .user-info { display: flex; align-items: center; gap: 16px; font-size: 13px; }
.header a          { color: #fff; text-decoration: none; }
.header a:hover    { text-decoration: underline; }

/* — ヘッダーのプルダウンメニュー — */
.menu-dropdown { position: relative; }
.menu-dropdown summary {
  display: flex; align-items: center; gap: 8px;
  padding: 7px 14px; border-radius: 20px;
  cursor: pointer; list-style: none; font-size: 13px; font-weight: 600;
  transition: background .15s;
}

.menu-dropdown summary::-webkit-details-marker,
.menu-dropdown summary::marker { display: none; content: ""; }

```



```

.menu-dropdown summary:hover,
.menu-dropdown[open] summary { background: rgba(255,255,255,.18); }
.menu-dropdown .caret { font-size: 10px; opacity: .8; }
.menu-dropdown .ico { margin-right: 0; }

.dropdown-panel {
  position: absolute; top: calc(100% + 8px); right: 0;
  background: #fff; border-radius: 8px; box-shadow: var(--shadow);
  min-width: 190px; overflow: hidden; z-index: 200;
}
.dropdown-panel a {
  display: flex; align-items: center; gap: 10px;
  padding: 12px 16px; color: var(--text) !important;
  font-size: 13px; font-weight: 600;
}
.dropdown-panel a:hover { background: var(--bg); text-decoration: none; }
.dropdown-panel a .ico { margin-right: 0; }

/* — コンテナ — */
.container { max-width: 1000px; margin: 28px auto; padding: 0 16px; }

/* — カード — */
.card {
  background: var(--card);
  border-radius: 8px;
  box-shadow: var(--shadow);
  padding: 24px;
  margin-bottom: 20px;
}
.card h2 {
  color: var(--teal);
  font-size: 17px;
  margin-bottom: 18px;
  padding-bottom: 10px;
  border-bottom: 2px solid var(--teal-light);
}

/* — ボタン — */
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 6px;
  padding: 9px 22px;
  border: none;
  border-radius: 5px;
  font-size: 14px;
  font-weight: 600;
  cursor: pointer;
  text-decoration: none;
  transition: background .15s, box-shadow .15s;
  vertical-align: middle;
}
.btn:active { transform: translateY(1px); }

.btn-primary { background: var(--teal); color: #fff; }
.btn-primary:hover { background: var(--teal-dark); }

.btn-secondary { background: #607D8B; color: #fff; }
.btn-secondary:hover { background: #455A64; }

.btn-green { background: var(--green); color: #fff; }
.btn-green:hover { background: var(--green-dk); }

```

```

.btn-danger { background: var(--red); color: #fff; }
.btn-danger:hover { background: var(--red-dk); }

.btn-outline {
  background: transparent;
  border: 2px solid var(--teal);
  color: var(--teal);
}
.btn-outline:hover { background: var(--teal); color: #fff; }

.btn-sm { padding: 5px 12px; font-size: 12px; }
.btn-lg { padding: 12px 32px; font-size: 16px; }

/* アイコン */
.ico { display: inline-block; width: 24px; height: 24px;
  border-radius: 50%; vertical-align: middle; margin-right: 8px; }
.ico-lg { width: 40px; height: 40px; border-radius: 50%; vertical-align: middle; margin-right: 10px; }

/* — アラート — */
.alert {
  padding: 12px 18px;
  border-radius: 5px;
  margin-bottom: 18px;
  font-size: 14px;
  font-weight: 500;
}
.alert-success { background: #E8F5E9; color: #2E7D32; border-left: 4px solid var(--green); }
.alert-danger { background: #FFEBEE; color: #B71C1C; border-left: 4px solid var(--red); }
.alert-info { background: #E3F2FD; color: #1565C0; border-left: 4px solid #42A5F5; }

/* — テーブル — */
.table { width: 100%; border-collapse: collapse; font-size: 13px; }
.table th {
  background: var(--teal);
  color: #fff;
  padding: 10px 14px;
  text-align: left;
  font-weight: 600;
}
.table td { padding: 9px 14px; border-bottom: 1px solid #ECEFF1; }
.table tbody tr:hover td { background: #F5F5F5; }
.table input[type=text] {
  width: 100%;
  padding: 4px 8px;
  border: 1px solid #CFD8DC;
  border-radius: 3px;
  font-size: 13px;
}

/* — フォーム — */
.form-group { margin-bottom: 18px; }
.form-group label { display: block; font-size: 12px; font-weight: 700; color: var(--text-muted); margin-bottom: 5px; }
.form-group input[type=text],
.form-group input[type=password],
.form-group select,
.form-group textarea {
  width: 100%;
  padding: 9px 12px;
  border: 1px solid #CFD8DC;
  border-radius: 4px;
  font-size: 14px;
}

```

```

    outline: none;
    transition: border-color .2s;
}
.form-group input:focus,
.form-group select:focus { border-color: var(--teal); box-shadow: 0 0 0 3px rgba(0,137,123,.15); }

.filter-row { display: flex; gap: 12px; align-items: flex-end; flex-wrap: wrap; margin-bottom: 18px; }
.filter-row .form-group { margin-bottom: 0; flex: 1; min-width: 130px; }

/* — ログイン画面 — */
.login-page {
    min-height: 100vh;
    display: flex;
    align-items: center;
    justify-content: center;
    background: linear-gradient(135deg, var(--teal-dark) 0%, var(--teal-light) 100%);
}
.login-card {
    background: #fff;
    border-radius: 12px;
    padding: 44px 40px;
    width: 100%;
    max-width: 400px;
    box-shadow: 0 8px 32px rgba(0,0,0,.22);
}
.login-title { text-align: center; font-size: 30px; font-weight: 700; color: var(--teal); letter-spacing: 2px; }
.login-subtitle { text-align: center; font-size: 12px; color: var(--text-muted); margin-bottom: 32px; }

/* — メニューグリッド — */
.menu-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
    gap: 14px;
    margin-top: 16px;
}
.menu-btn {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 10px;
    padding: 22px 12px;
    background: #fff;
    border: 2px solid var(--teal-light);
    border-radius: 8px;
    text-decoration: none;
    color: var(--teal-dark);
    font-weight: 700;
    font-size: 13px;
    transition: all .15s;
    cursor: pointer;
    text-align: center;
}
.menu-btn:hover { background: var(--teal); color: #fff; border-color: var(--teal); }
.menu-btn:hover .ico-lg { background: rgba(255,255,255,.3); }

/* — 埋有率カード — */
.occ-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(190px, 1fr));
    gap: 16px;
}
.occ-card {
    background: #fff;

```

```

border-radius: 8px;
padding: 20px;
box-shadow: var(--shadow);
cursor: pointer;
transition: transform .15s, box-shadow .15s;
border-left: 4px solid var(--teal);
text-decoration: none;
color: inherit;
display: block;
}
.occ-card:hover { transform: translateY(-3px); box-shadow: 0 5px 18px rgba(0,0,0,.15); }
.occ-name { font-size: 16px; font-weight: 700; margin-bottom: 14px; color: var(--text); }
.occ-bar-bg { height: 8px; background: #ECEFF1; border-radius: 4px; overflow: hidden; margin-bottom: 8px; }
.occ-bar { height: 100%; border-radius: 4px; background: var(--green); }
.occ-bar.med { background: var(--yellow); }
.occ-bar.high { background: var(--orange); }
.occ-bar.full { background: var(--red); }
.occ-pct { font-size: 26px; font-weight: 700; color: var(--teal); }
.occ-label { font-size: 11px; color: var(--text-muted); }

/* — 座席グリッド — */
.seat-grid { display: flex; flex-wrap: wrap; gap: 10px; margin-top: 14px; }
.seat-btn {
  width: 76px;
  height: 54px;
  border: none;
  border-radius: 5px;
  font-size: 11px;
  font-weight: 700;
  cursor: pointer;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 3px;
  transition: opacity .15s, transform .1s;
  padding: 0 4px;
}
.seat-btn .seat-id { font-size: 13px; }
.seat-btn .seat-user { font-size: 10px; opacity: .85; overflow: hidden; white-space: nowrap;
  max-width: 70px; text-overflow: ellipsis; }
.seat-btn.available { background: var(--green); color: #fff; }
.seat-btn.available:hover { opacity: .85; transform: scale(1.05); }
.seat-btn.occupied { background: var(--red); color: #fff; cursor: not-allowed; }
.seat-btn.selected { background: var(--teal); color: #fff; outline: 3px solid var(--teal-dark); }

/* — ステージバー — */
.stage-bar { display: flex; border-radius: 5px; overflow: hidden; margin-bottom: 22px; }
.stage-step { flex: 1; padding: 9px 0; text-align: center; font-size: 12px; font-weight: 700;
  background: #ECEFF1; color: var(--text-muted); }
.stage-step.done { background: var(--teal-light); color: #fff; }
.stage-step.active { background: var(--teal); color: #fff; }

/* — パンくずリスト — */
.breadcrumb { display: flex; gap: 8px; align-items: center; font-size: 13px;
  color: var(--text-muted); margin-bottom: 14px; }
.breadcrumb a { color: var(--teal); text-decoration: none; }
.breadcrumb a:hover { text-decoration: underline; }

/* — サマリバツジ — */
.summary-row { display: flex; gap: 12px; margin-bottom: 16px; flex-wrap: wrap; }
.badge { padding: 4px 12px; border-radius: 12px; font-size: 12px; font-weight: 700; }
.badge-green { background: #E8F5E9; color: #2E7D32; }

```

```

.badge-red { background: #FFE0E0; color: #B71C1C; }
.badge-gray { background: #E0E0E0; color: #546E7A; }

/* — 2週間分の予約状況（折りたたみ行） — */
.day-summary { border: 1px solid #E0E0E0; border-radius: 6px; margin-bottom: 6px; overflow: hidden; }
.day-summary summary {
  display: flex; align-items: center; gap: 10px;
  padding: 10px 14px; cursor: pointer; list-style: none;
  font-size: 13px; transition: background .15s;
}
.day-summary summary::-webkit-details-marker, .day-summary summary::marker { display: none; content: ""; }
.day-summary summary:hover { background: #F5F5F5; }
.day-summary[open] summary { background: #E0F2F1; }
.day-summary .day-date { font-weight: 700; min-width: 110px; }
.day-summary .caret { margin-left: auto; font-size: 10px; color: #9E9E9E; }
.day-detail { padding: 12px 14px; border-top: 1px solid #E0E0E0; background: #FAFAFA; }

/* — レイアウトプレースホルダー — */
.layout-placeholder {
  width: 100%;
  height: 140px;
  border: 2px dashed #CFD8DC;
  border-radius: 5px;
  display: flex;
  align-items: center;
  justify-content: center;
  color: var(--text-muted);
  font-size: 13px;
  margin: 16px 0;
  background: #FAFAFA;
}

/* — 管理者バッジ — */
.admin-badge {
  background: #FF9800;
  color: #fff;
  padding: 2px 8px;
  border-radius: 4px;
  font-size: 11px;
  font-weight: 700;
  margin-left: 6px;
}

/* — チェックボックスグループ — */
.check-group { display: flex; flex-wrap: wrap; gap: 10px; }
.check-group label { display: flex; align-items: center; gap: 5px; cursor: pointer; }

/* 社員検索結果（代理予約） */
.account-row { flex: 0 0 auto; padding: 8px 14px; border: 1px solid #E0E0E0; border-radius: 6px; background: #fff; }
.account-row:hover { background: #F5F5F5; }

/* — 行先選択ボタン — */
.gyosaki-group { display: flex; gap: 14px; flex-wrap: wrap; margin-top: 8px; }
.gyosaki-btn {
  flex: 1; min-width: 120px;
  padding: 24px 12px;
  font-size: 18px;
  font-weight: 700;
  text-align: center;
  border-radius: 8px;
  border: 3px solid transparent;
  cursor: pointer;
  transition: all .15s;
}

```

```
}
.gyosaki-btn.office { background: #E3F2FD; color: #1565C0; border-color: #90CAF9; }
.gyosaki-btn.office:hover { background: #1565C0; color: #fff; }
.gyosaki-btn.remote { background: #E8F5E9; color: #2E7D32; border-color: #A5D6A7; }
.gyosaki-btn.remote:hover { background: #2E7D32; color: #fff; }
.gyosaki-btn.trip { background: #FFF3E0; color: #E65100; border-color: #FFCC80; }
.gyosaki-btn.trip:hover { background: #E65100; color: #fff; }

/* — レスポンシブ補正 — */
@media (max-width: 600px) {
  .container { padding: 0 10px; }
  .occ-grid { grid-template-columns: 1fr 1fr; }
  .menu-grid { grid-template-columns: 1fr 1fr; }
}
```